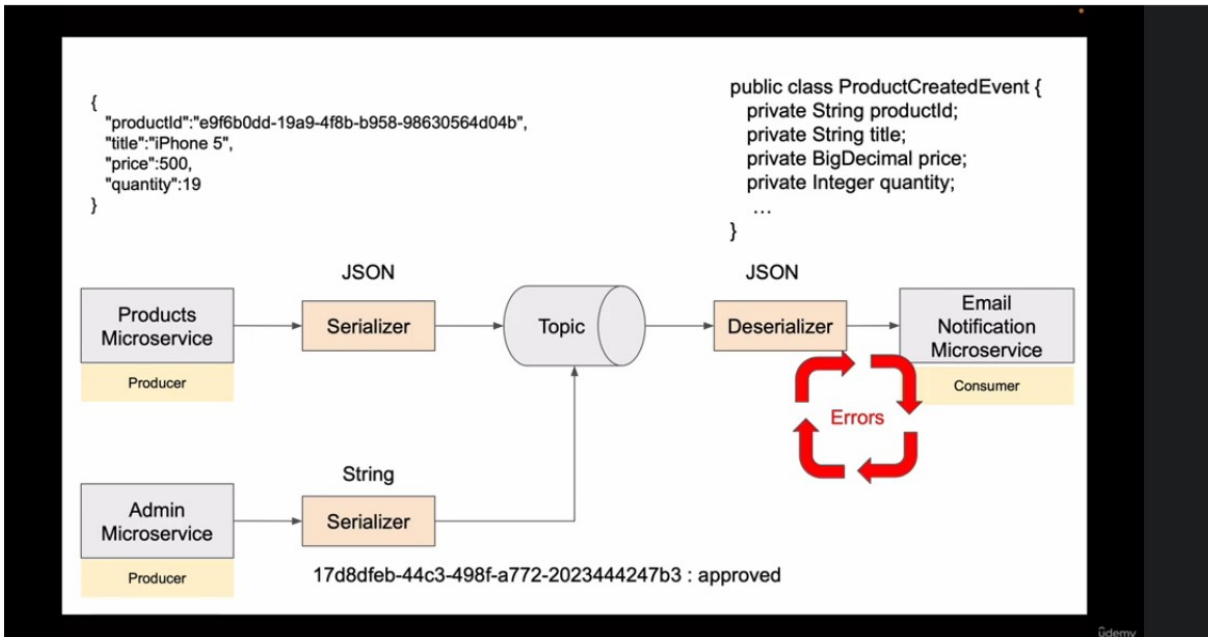
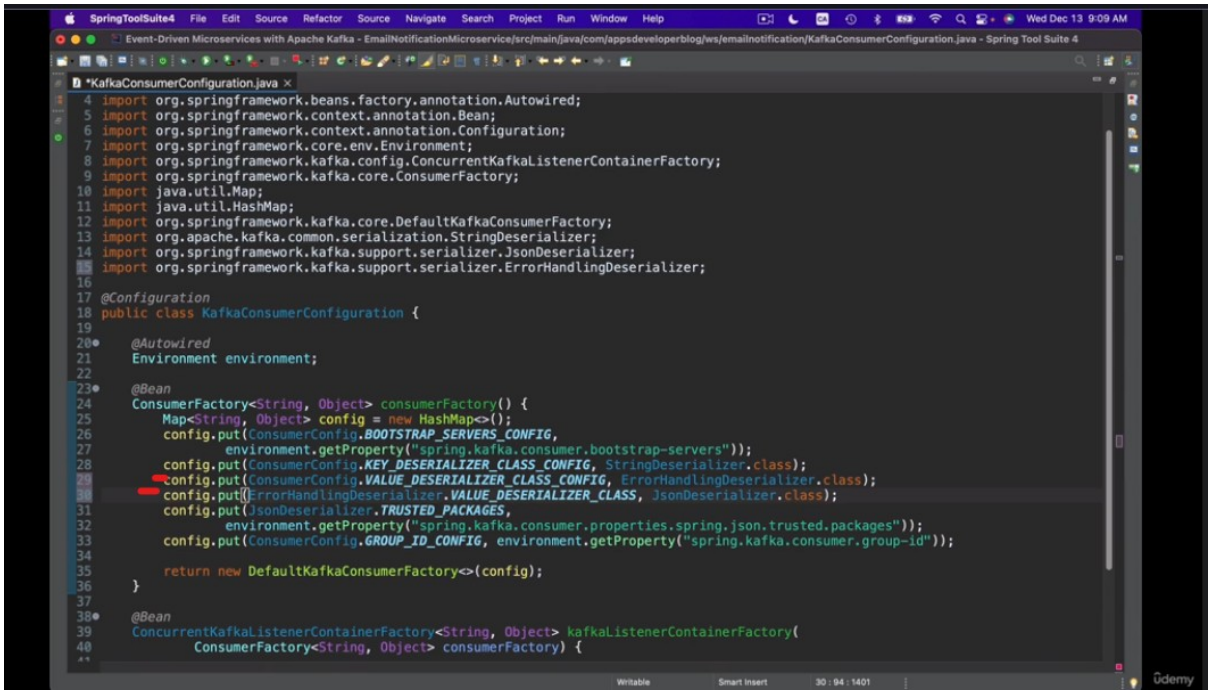
KAFKA CONSUMER HANDLE DESERIALIZER ERROR:



WE SHOULD USE ERROR HANDLING DESERIALIZER CLASS SO THAT IF IN TOPIC SOME OTHER EVENT COMES WHICH DOES NOT FIT THE STRUCTURE THAT THE CONSUMER IS CONSUMING THEN IT SHOULD HANDLE IT (SKIP) INSTEAD OF THROWING ERROR



Solution: ErrorHandlingDeserializer

Very common production config.

Java

```java
props.put(
    ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
    ErrorHandlingDeserializer.class);

props.put(
    ErrorHandlingDeserializer.VALUE_DESERIALIZER_CLASS,
    JsonDeserializer.class);
```

Flow:

Plain Text

```
Kafka Record
     ↓
ErrorHandlingDeserializer
     ↓
JsonDeserializer
     ↓
Success -> Listener

or

Failure -> ErrorHandler / DLT
```

Error Handler

```java
@Bean
public DefaultErrorHandler errorHandler() {

    FixedBackOff backOff =
        new FixedBackOff(2000L, 3);

    return new DefaultErrorHandler(backOff);
}
```

Meaning:

```
Failure
   ↓
Retry 1
   ↓
Retry 2
   ↓
Retry 3
   ↓
Give Up
```

Send Failed Message to DLT

```java
@Bean
public DefaultErrorHandler errorHandler(
        KafkaTemplate<Object, Object> template) {

    DeadLetterPublishingRecoverer recoverer =
        new DeadLetterPublishingRecoverer(template);

    return new DefaultErrorHandler(
            recoverer,
            new FixedBackOff(2000L, 3));
}
```

Flow:

```
orders-topic
      ↓
Processing failed
      ↓
Retry 3 times
      ↓
orders-topic.DLT
```

Very common production setup.

Configure in Listener Container Factory

```java
@Bean
public ConcurrentKafkaListenerContainerFactory<String, OrderEvent>
kafkaListenerContainerFactory(
        ConsumerFactory<String, OrderEvent> cf,
        DefaultErrorHandler errorHandler) {

    ConcurrentKafkaListenerContainerFactory<String, OrderEvent> factory =
            new ConcurrentKafkaListenerContainerFactory<>();

    factory.setConsumerFactory(cf);

    factory.setCommonErrorHandler(errorHandler);

    factory.setConcurrency(3);

    return factory;
```

Show more lines

Real Production Factory

```java
public ConcurrentKafkaListenerContainerFactory<String, OrderEvent>
kafkaListenerContainerFactory(
        ConsumerFactory<String, OrderEvent> cf,
        DefaultErrorHandler errorHandler) {

    ConcurrentKafkaListenerContainerFactory<String, OrderEvent> factory =
            new ConcurrentKafkaListenerContainerFactory<>();

    factory.setConsumerFactory(cf);

    factory.setConcurrency(3);

    factory.setCommonErrorHandler(errorHandler);

    factory.getContainerProperties()
           .setAckMode(AckMode.MANUAL);
```

Show more lines

This factory controls:

✓ **Number of consumers (concurrency)**

✓ **Retry**

✓ **Backoff**

✓ **DLT**

✓ **Manual Ack**

✓ **Error Handling**

✓ **Deserialization behaviour**

✓ **Batch processing**

That's why when someone says:

> "Why do we need KafkaListenerContainerFactory?"

A strong interview answer is:

> "It creates the listener containers behind `@KafkaListener` and centralizes configuration such as concurrency, acknowledgment mode, deserialization, retries, error handling, backoff policies, and dead-letter-topic publishing."

DLT (DEAD LETTER TOPIC)

Yes, **a DLT (Dead Letter Topic) must exist as a Kafka topic**, but whether you create it manually or automatically depends on your setup.

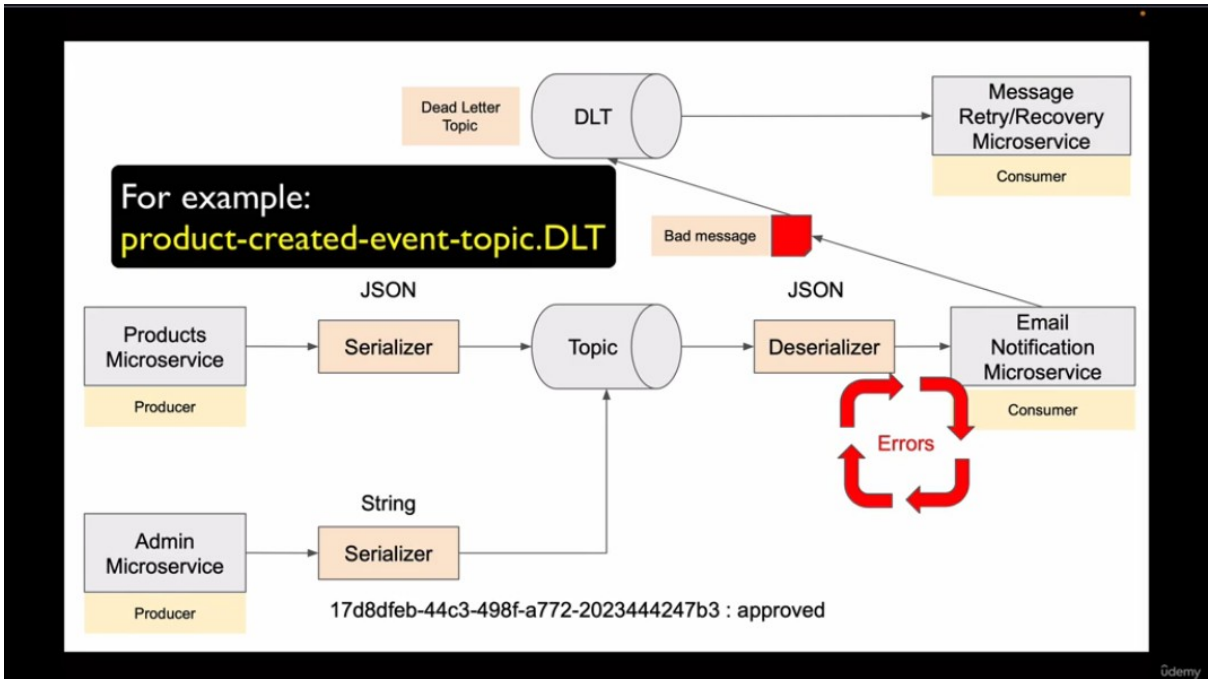
Typical Flow

```
orders-topic
      |
      v
Consumer
      |
      | fails after retries
      v
orders-topic.DLT
```

or

```
orders-topic-dlt
```

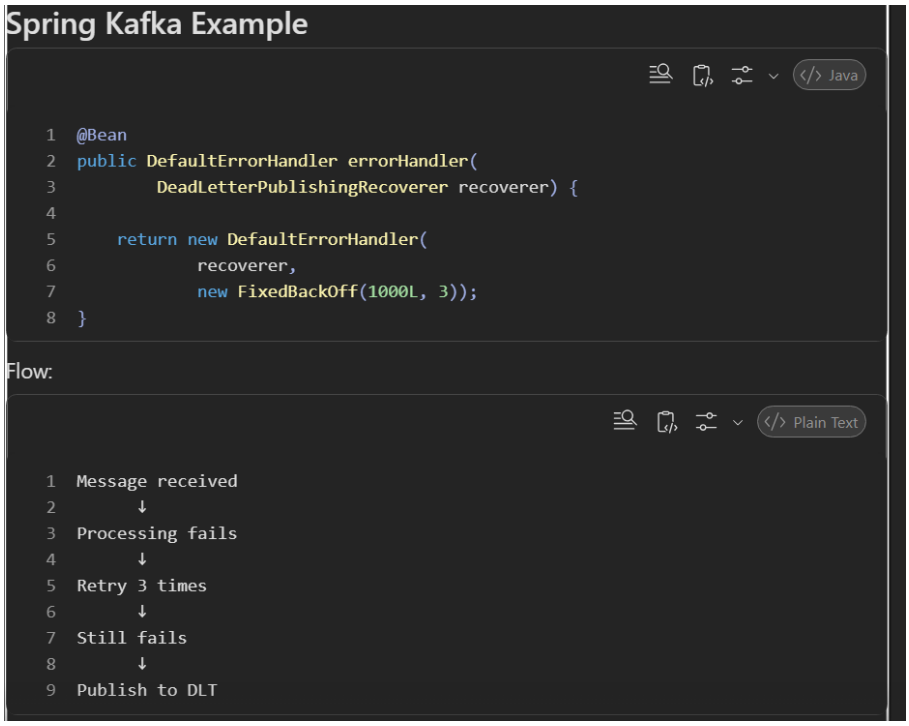
depending on your naming strategy.

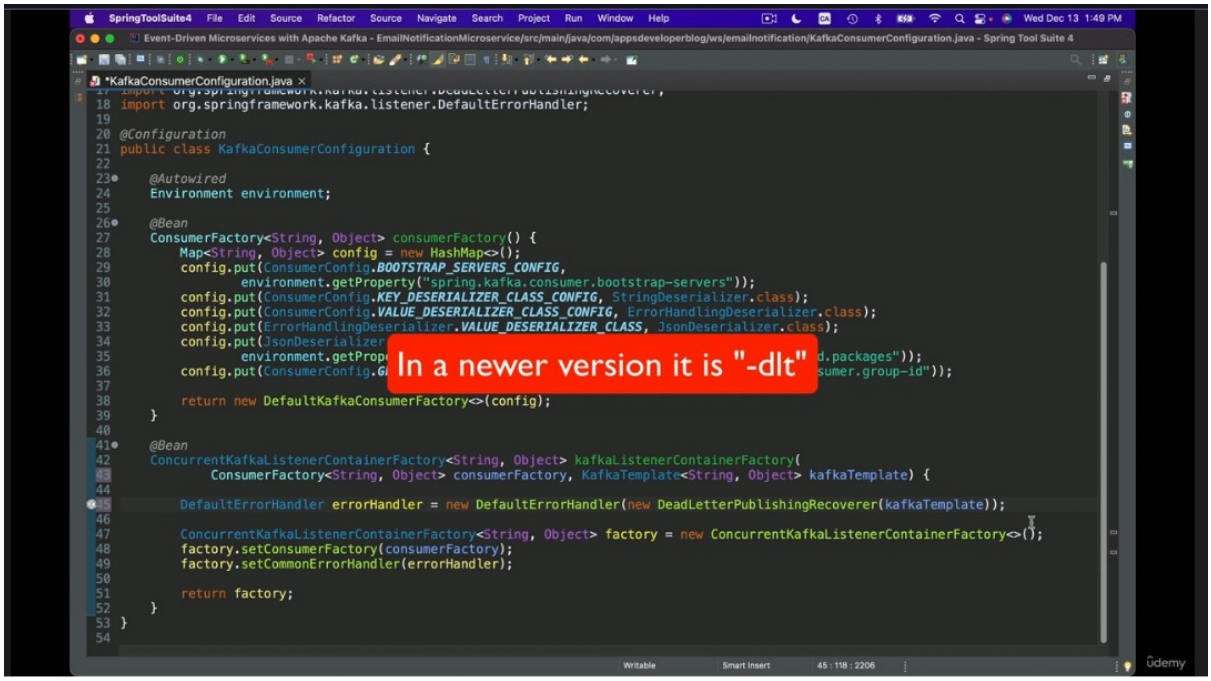


A Dead Letter Topic is a separate Kafka topic where messages are sent after retry attempts are exhausted.

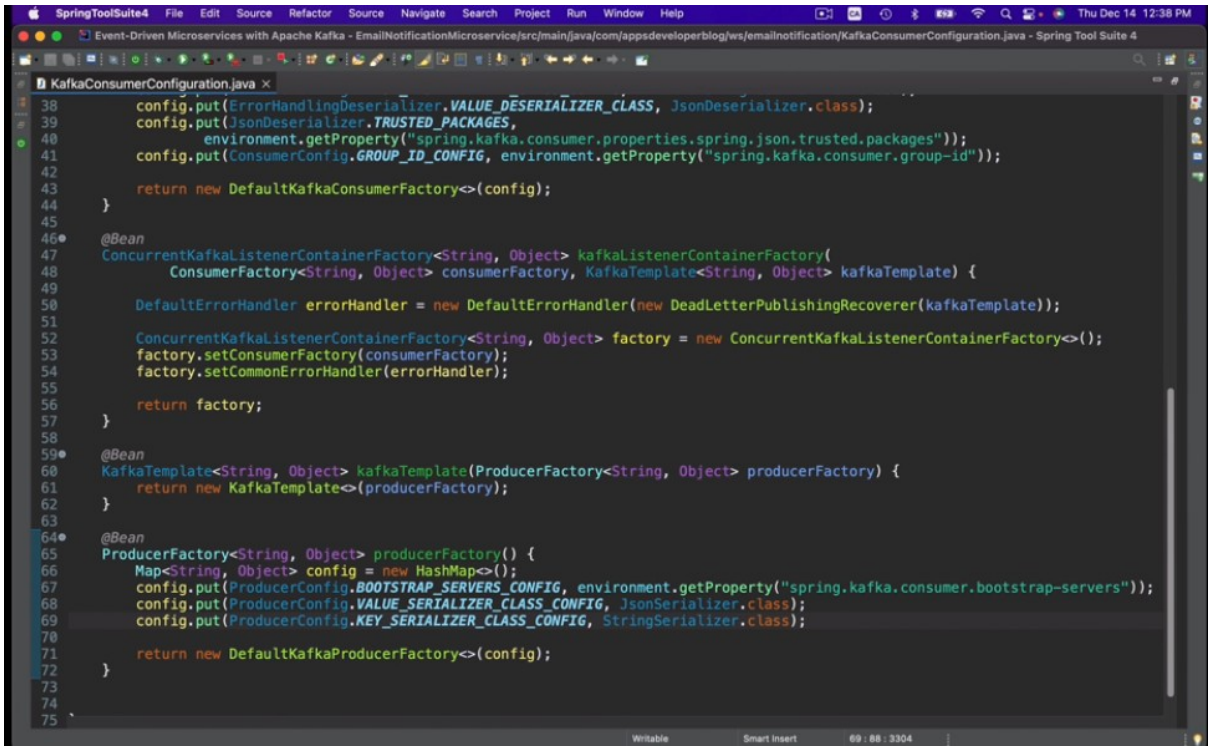
In production, DLT topics are usually created manually by infrastructure teams with the appropriate partition count and replication factor.

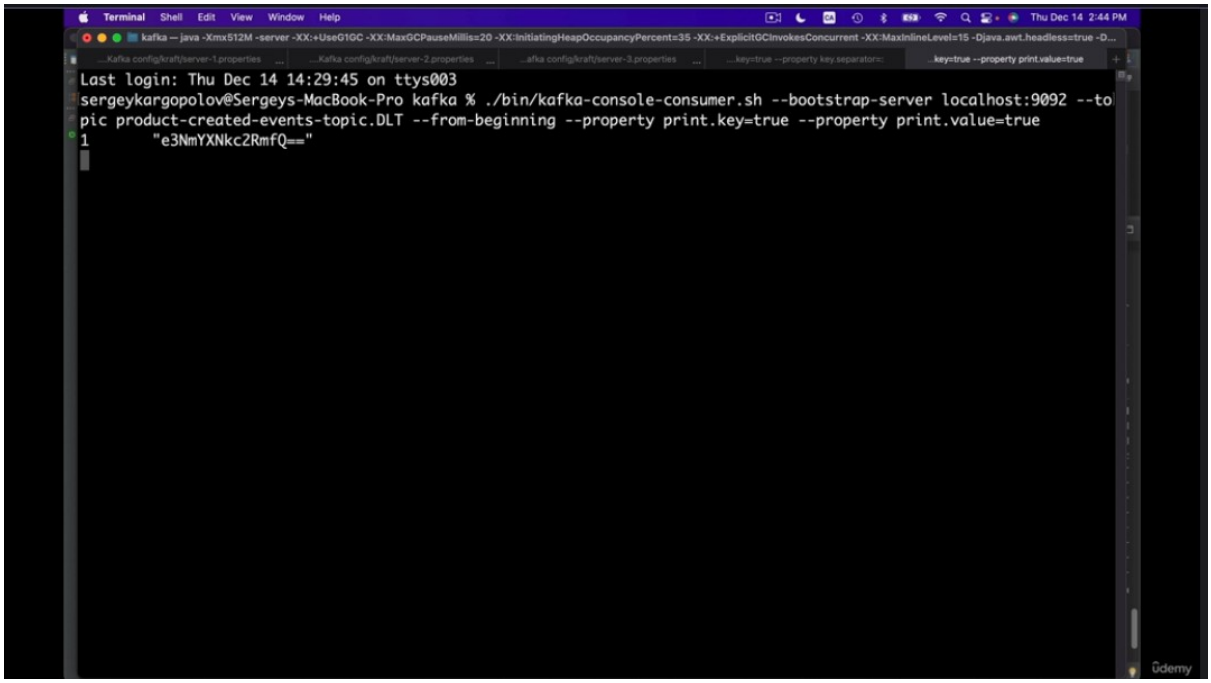
Spring Kafka can automatically publish failed records to the DLT using DeadLetterPublishingRecoverer and DefaultErrorHandler. A common naming convention is <topic>-dlt or <topic>.DLT.



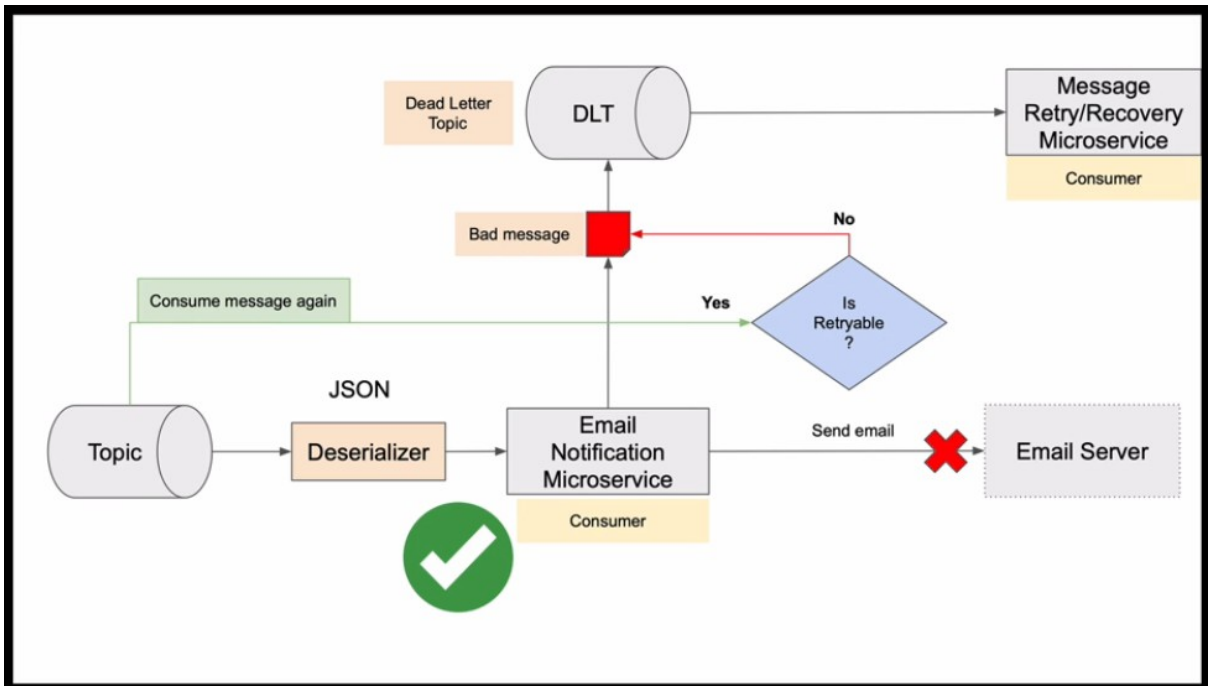


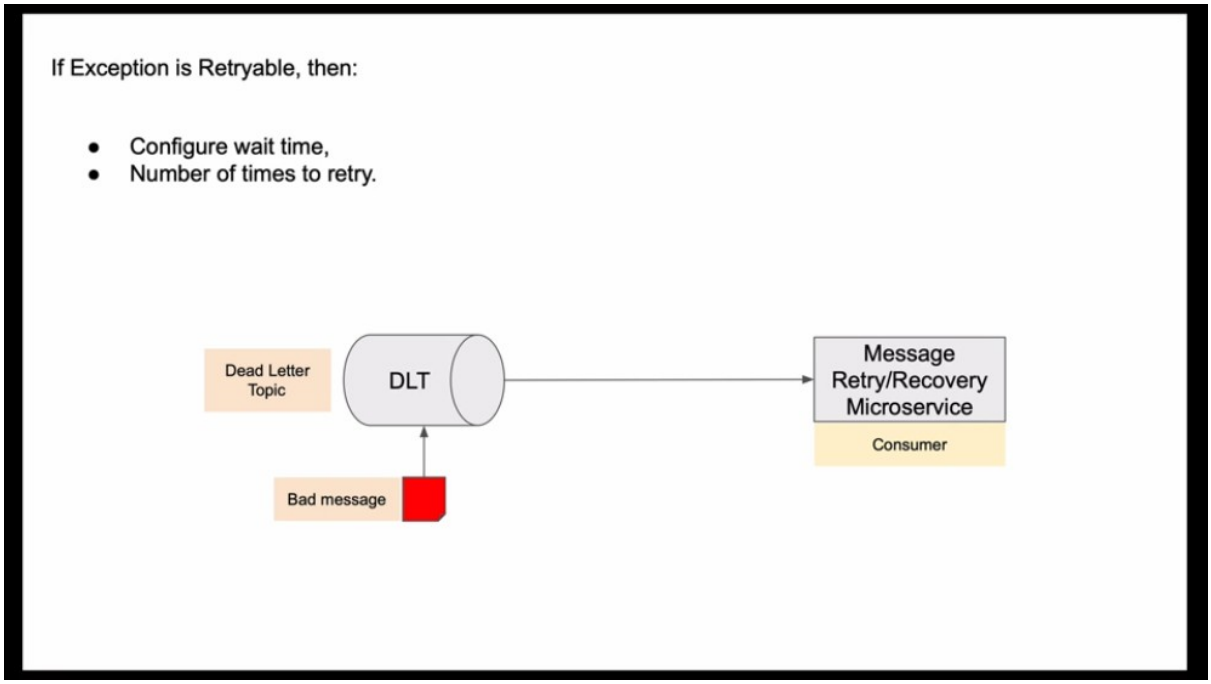
ADD KAFKA TEMPLATE & PRODUCER FACTORY BEAN FOR THE DEAD LETTER PUBLISHING RECOVERER





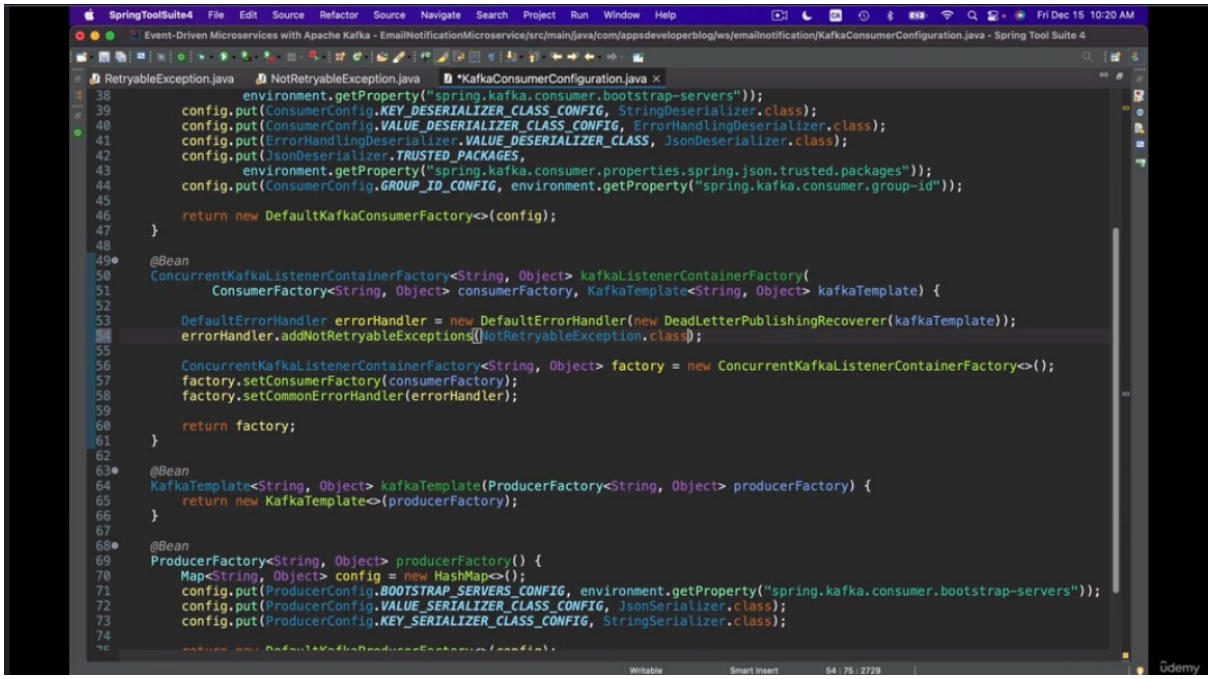
KAFKA CONSUMER EXCEPTION & RETRIES:

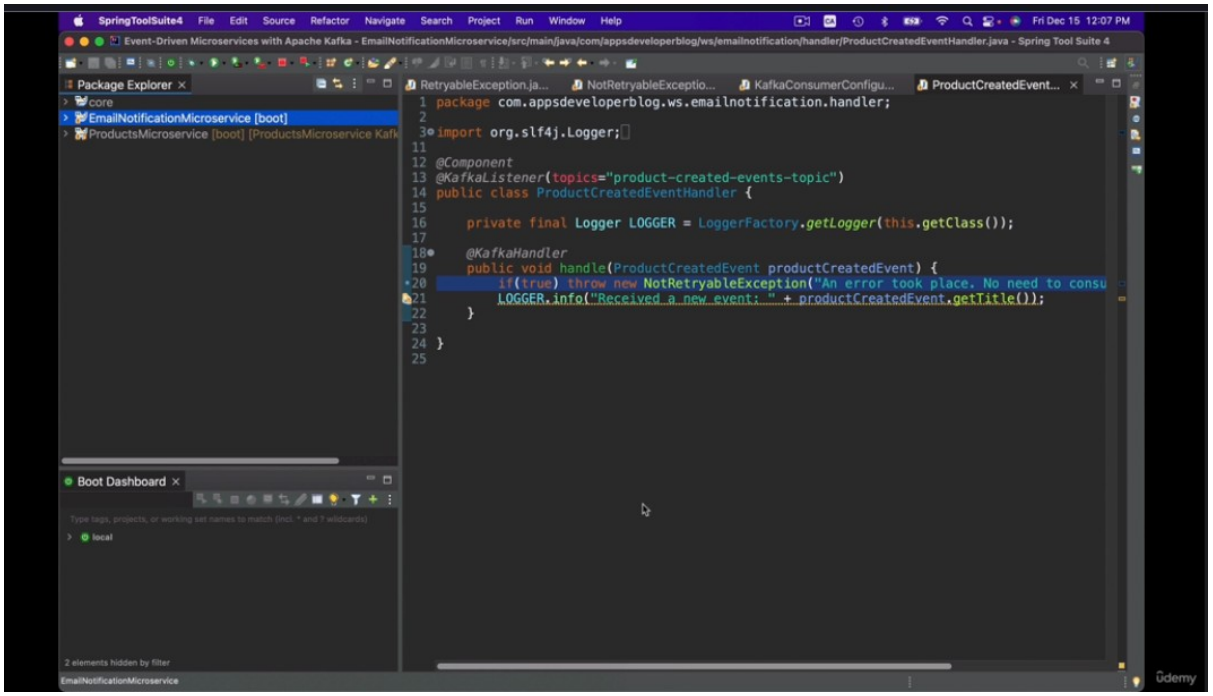




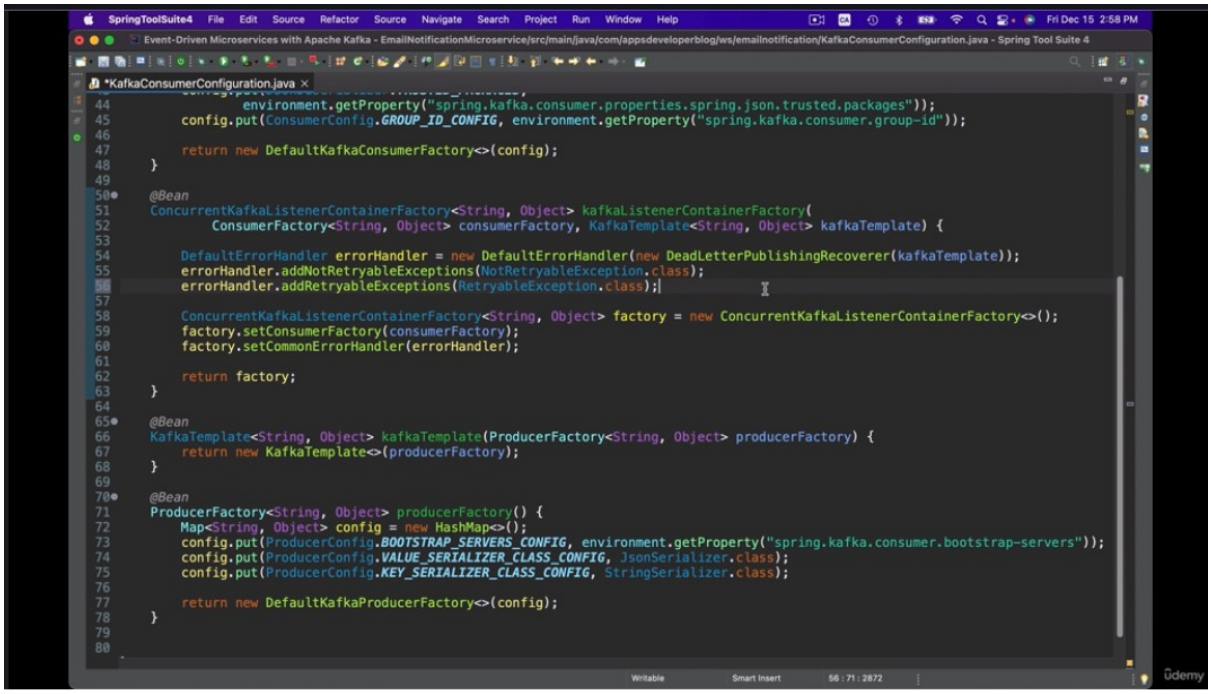
Handling Non-Retryable exception or Retryable exception

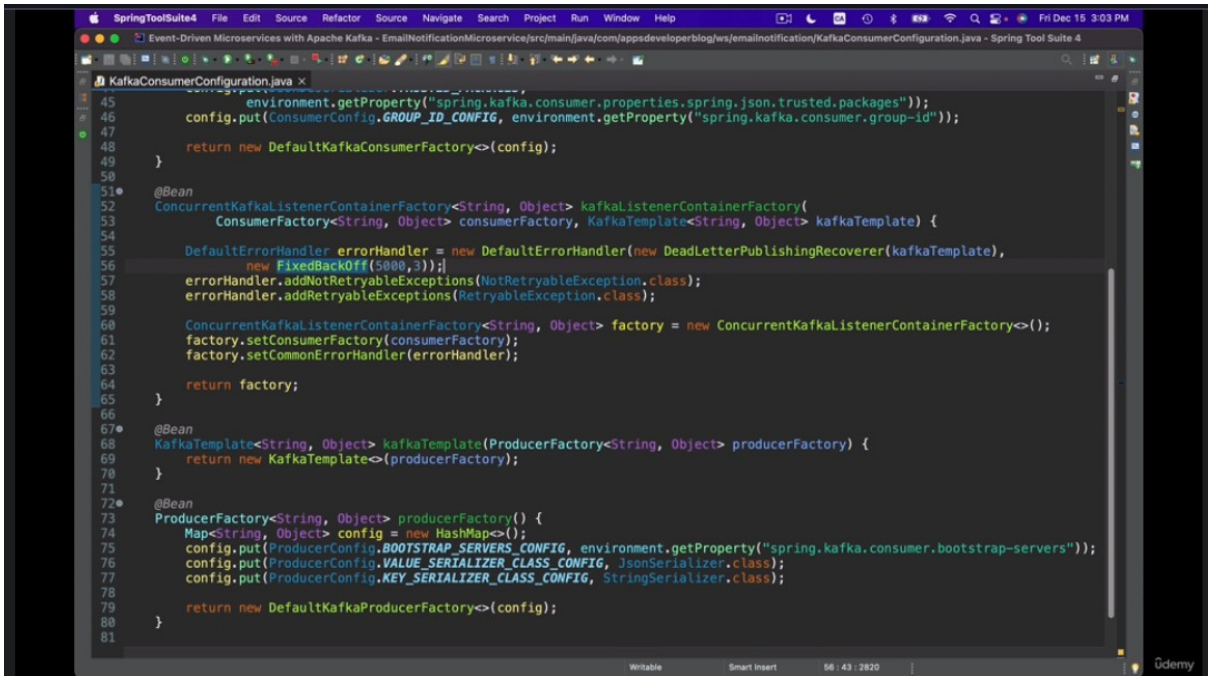
We can add extra addNoRetryableException by [,] separator


SpringToolSuite4 File Edit Source Refactor Source Navigate Search Project Run Window Help
Fri Dec 15 10:20 AM
Event-Driven Microservices with Apache Kafka - EmailNotificationMicroservice/src/main/java/com/appsdeveloperblog/ws/emailnotification/KafkaConsumerConfiguration.java - Spring Tool Suite 4
RetryableException.java
NotRetryableException.java
*KafkaConsumerConfiguration.java
environment.getProperty("spring.kafka.consumer.bootstrap-servers"));
config.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class);
config.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, ErrorHandlingDeserializer.class);
config.put(ErrorHandlingDeserializer.VALUE_DESERIALIZER_CLASS, JsonDeserializer.class);
config.put(JsonDeserializer.TRUSTED_PACKAGES,
environment.getProperty("spring.kafka.consumer.properties.spring.json.trusted.packages"));
config.put(ConsumerConfig.GROUP_ID_CONFIG, environment.getProperty("spring.kafka.consumer.group-id"));
return new DefaultKafkaConsumerFactory<>(config);
}
@Bean
ConcurrentKafkaListenerContainerFactory<String, Object> kafkaListenerContainerFactory(
ConsumerFactory<String, Object> consumerFactory, KafkaTemplate<String, Object> kafkaTemplate) {
DefaultErrorHandler errorHandler = new DefaultErrorHandler(new DeadLetterPublishingRecoverer(kafkaTemplate));
errorHandler.addNotRetryableExceptions(NotRetryableException.class);
ConcurrentKafkaListenerContainerFactory<String, Object> factory = new ConcurrentKafkaListenerContainerFactory<>();
factory.setConsumerFactory(consumerFactory);
factory.setCommonErrorHandler(errorHandler);
return factory;
}
@Bean
KafkaTemplate<String, Object> kafkaTemplate(ProducerFactory<String, Object> producerFactory) {
return new KafkaTemplate<>(producerFactory);
}
@Bean
ProducerFactory<String, Object> producerFactory() {
Map<String, Object> config = new HashMap<>();
config.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, environment.getProperty("spring.kafka.consumer.bootstrap-servers"));
config.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, JsonSerializer.class);
config.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
Writable
Smart Insert
54 : 75 : 2729
Udemy


SpringToolSuite4 File Edit Source Refactor Navigate Search Project Run Window Help
Fri Dec 15 12:07 PM
Event-Driven Microservices with Apache Kafka - EmailNotificationMicroservice/src/main/java/com/appsdeveloperblog/ws/emailnotification/handler/ProductCreatedEventHandler.java - Spring Tool Suite 4
Package Explorer
core
EmailNotificationMicroservice [boot]
ProductsMicroservice [boot]
RetryableException.ja...
NotRetryableExceptio...
KafkaConsumerConfigu...
ProductCreatedEvent...
package com.appsdeveloperblog.ws.emailnotification.handler;
import org.slf4j.Logger;
@Component
@KafkaListener(topics="product-created-events-topic")
public class ProductCreatedEventHandler {
private final Logger LOGGER = LoggerFactory.getLogger(this.getClass());
@KafkaHandler
public void handle(ProductCreatedEvent productCreatedEvent) {
if(true) throw new NotRetryableException("An error took place. No need to consu
LOGGER.info("Received a new event: " + productCreatedEvent.getTitle());
}
}
Boot Dashboard
local
2 elements hidden by filter
EmailNotificationMicroservice
Udemy

RETRYABLE EXCEPTION:





Real Production Setup

```java
@Bean
public DefaultErrorHandler errorHandler(
        KafkaTemplate<Object, Object> template) {

    DeadLetterPublishingRecoverer recoverer =
            new DeadLetterPublishingRecoverer(template);

    DefaultErrorHandler handler =
            new DefaultErrorHandler(
                    recoverer,
                    new FixedBackOff(2000L, 3));

    handler.addNotRetryableExceptions(
            ValidationException.class,
            IllegalArgumentException.class);
```

Show more lines

EXPONENTIAL BACKOFF WITH RETRIES

Example:

```java
@Bean
public DefaultErrorHandler errorHandler(
        KafkaTemplate<Object, Object> template) {

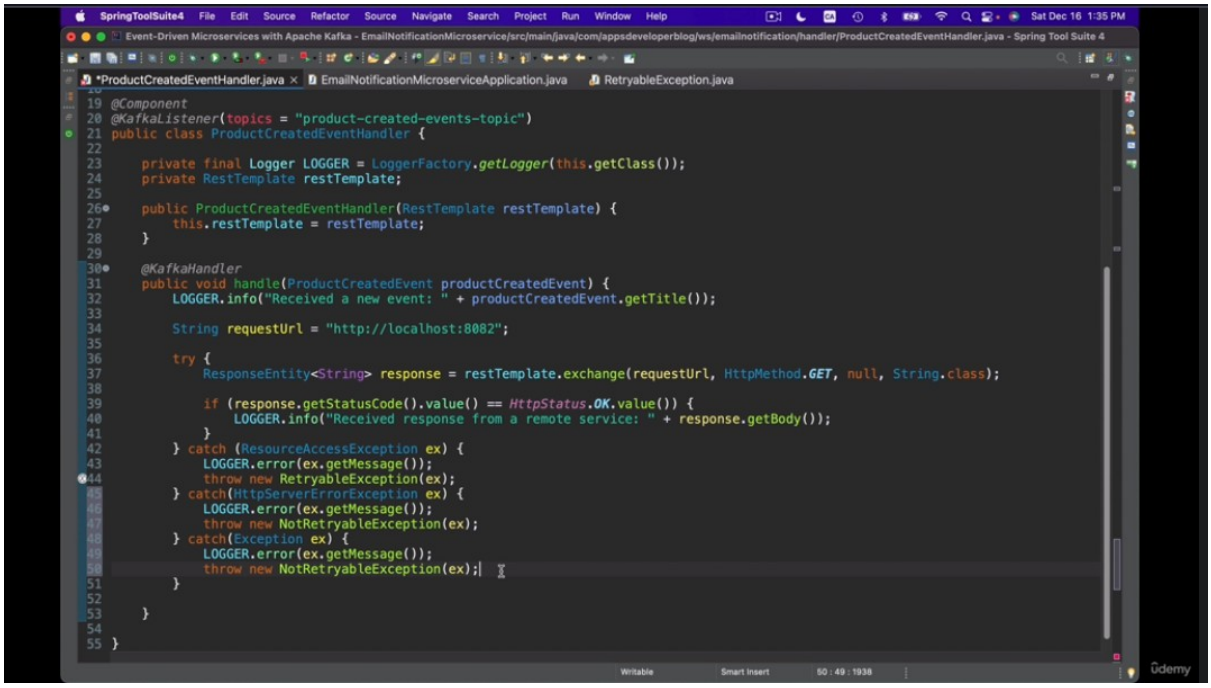
    DeadLetterPublishingRecoverer recoverer =
            new DeadLetterPublishingRecoverer(template);

    ExponentialBackOffWithMaxRetries backOff =
            new ExponentialBackOffWithMaxRetries(5);

    backOff.setInitialInterval(1000L); // 1 sec
    backOff.setMultiplier(2.0);        // double each time
    backOff.setMaxInterval(30000L);    // max 30 sec

    return new DefaultErrorHandler(recoverer, backOff);
}
```

Show more lines



Retrying won't help because the data itself is bad.

Plain Text

```
Message
   ↓
Validation Error
   ↓
No Retry
   ↓
DLT
```

Reason for DLT:

- Keep the failed message for investigation
- Don't block partition processing
- Allow support team to inspect later

```
Message
   ↓
DB Timeout
   ↓
Retry 1
   ↓
Retry 2
   ↓
Retry 3
   ↓
Still Failing
   ↓
DLT
```

Now your question is:

> "If it already retried 3 times and still goes to DLT, what's the point?"

Because Kafka doesn't know whether:

```
DB will be back in 1 min
DB will be back in 1 hour
DB will be back tomorrow
```

You can't retry forever.

DEBUGGING:::

```java
@KafkaListener(topics = "orders")
public void consume(
        OrderEvent event,
        @Header(KafkaHeaders.RECEIVED_TOPIC) String topic,
        @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
        @Header(KafkaHeaders.OFFSET) long offset) {

    System.out.println(topic);
    System.out.println(partition);
    System.out.println(offset);
}
```

Even Easier: Consume ConsumerRecord

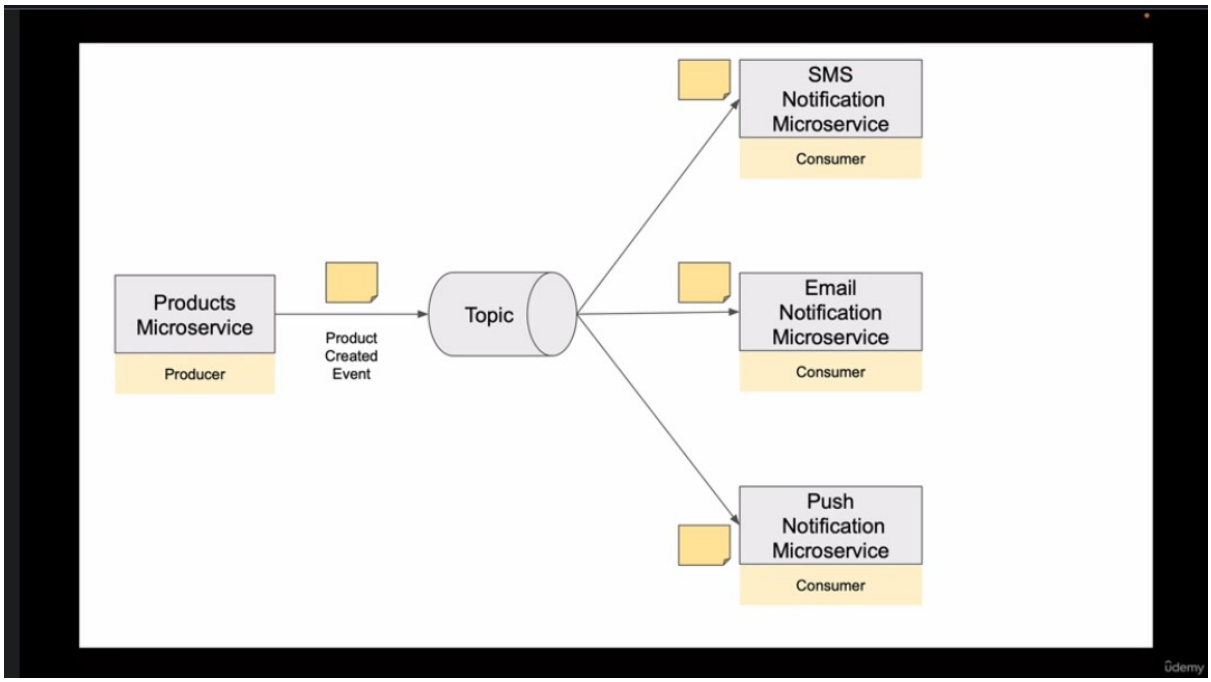
Instead of individual headers:

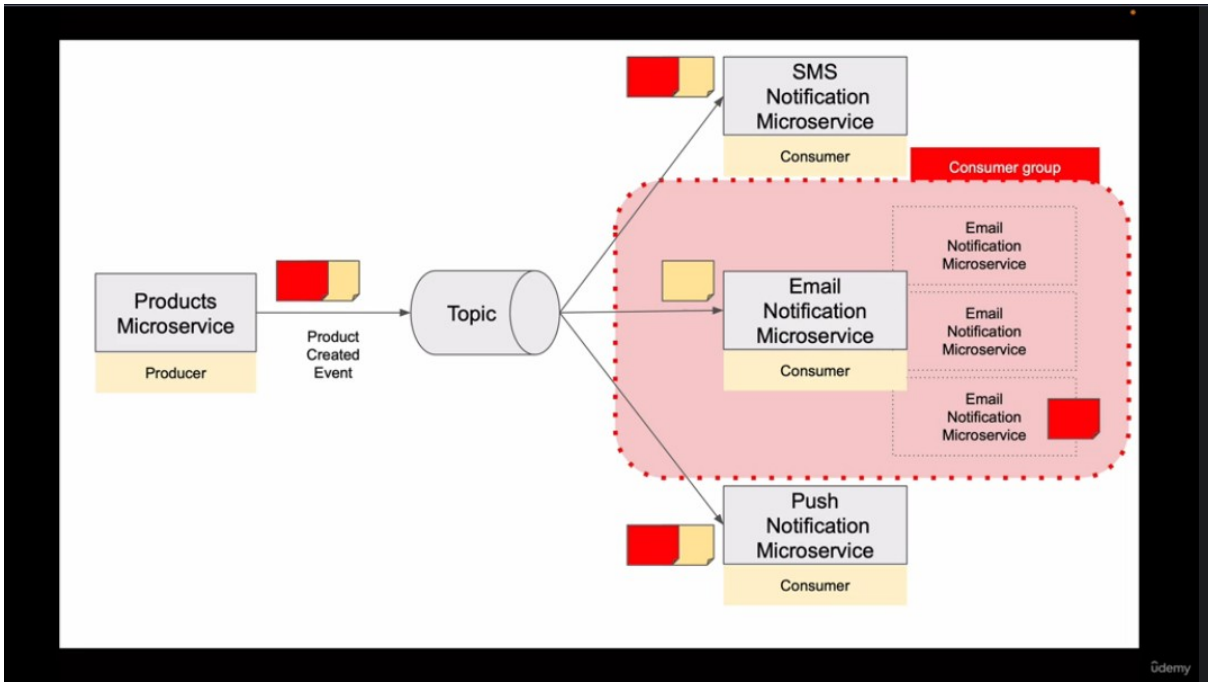
Java

```java
@KafkaListener(topics = "orders")
public void consume(
        ConsumerRecord<String, OrderEvent> record) {

    System.out.println(record.topic());
    System.out.println(record.partition());
    System.out.println(record.offset());
    System.out.println(record.key
    System.out.println(record.value());
}
```

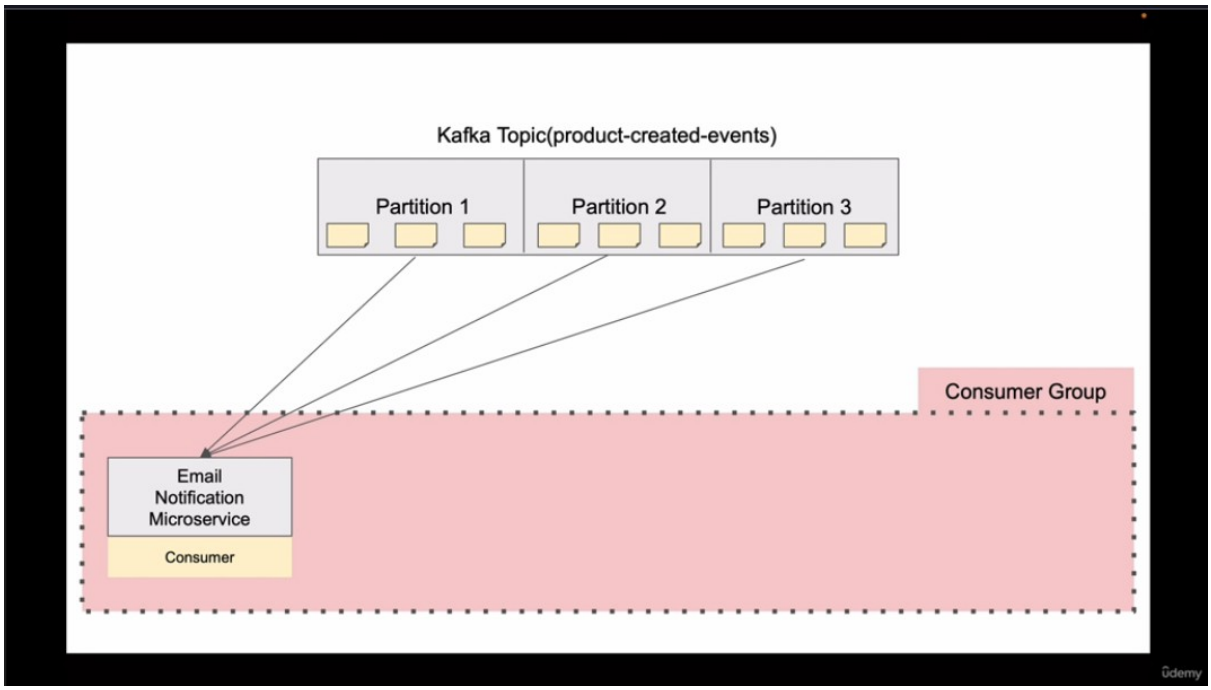
CONSUMER GROUP::



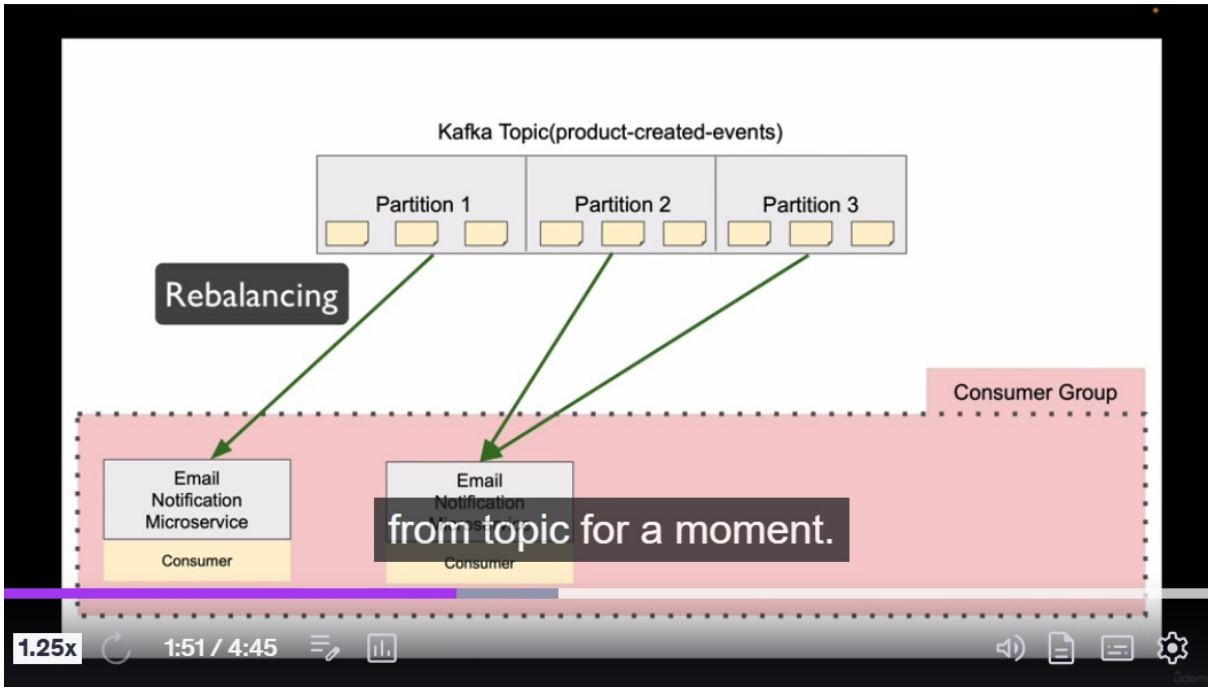


REBALANCING & PARTITION ASSIGNMENT IN APACHE KAFKA

If we have only one consumer instance. All partitions will come to one consumer.

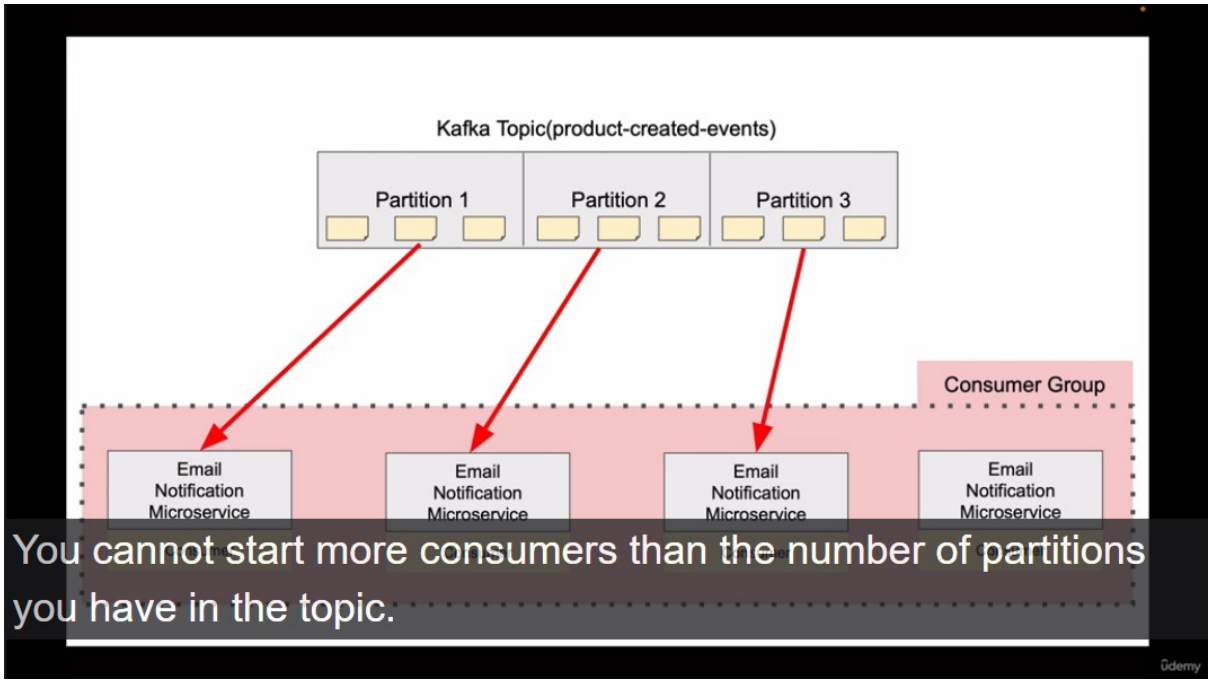


If you need to scale out then Kafka reassign the partitions among the consumer instances. While re-assigning consumer stop consuming messages for a moment

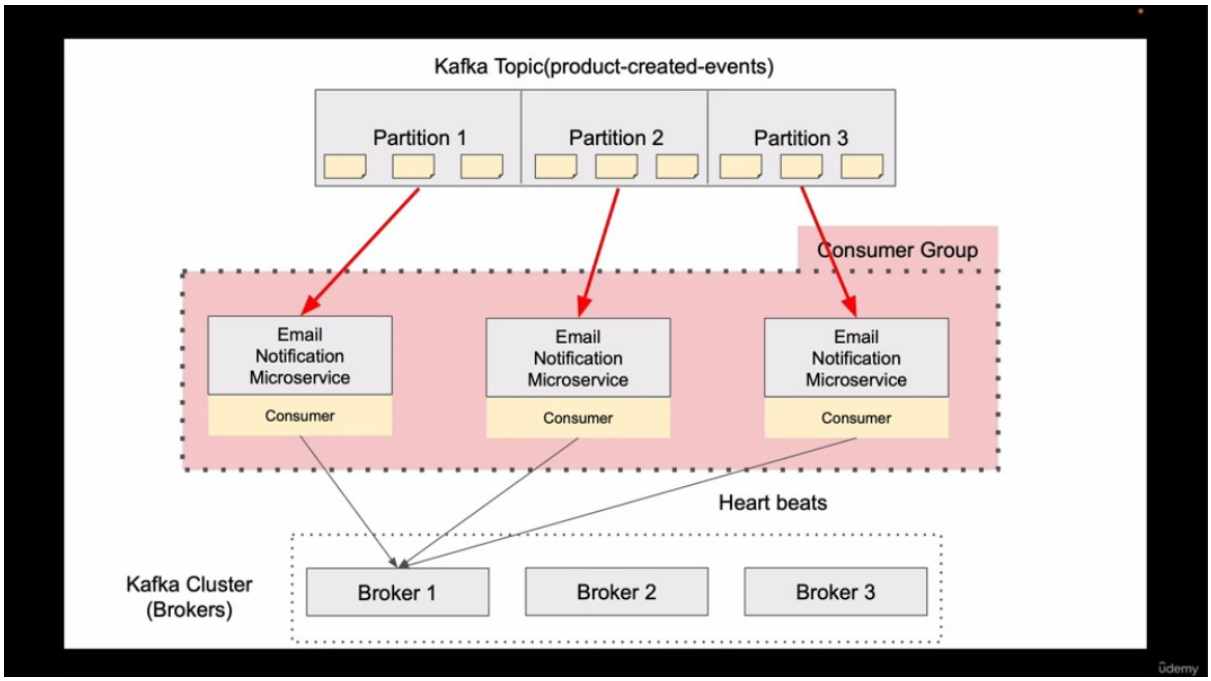


Rebalancing happens like one consumer is added or removed or even if partitions are added.

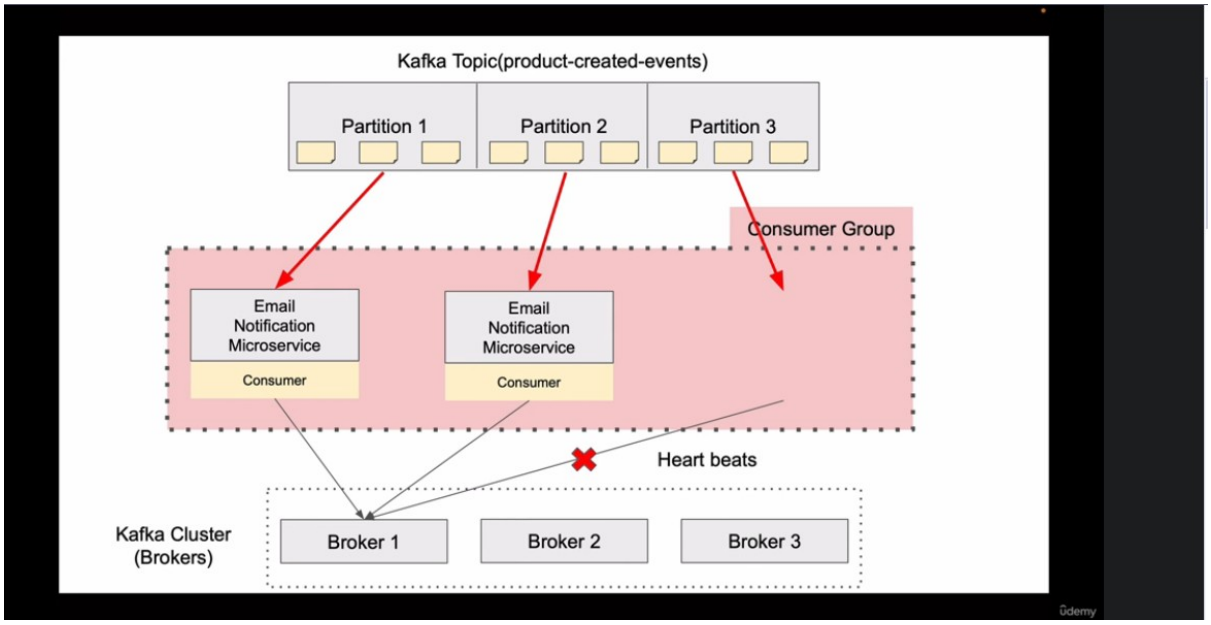
If you have more consumer then it will remain idle we cannot assign one partition to more than on consumer

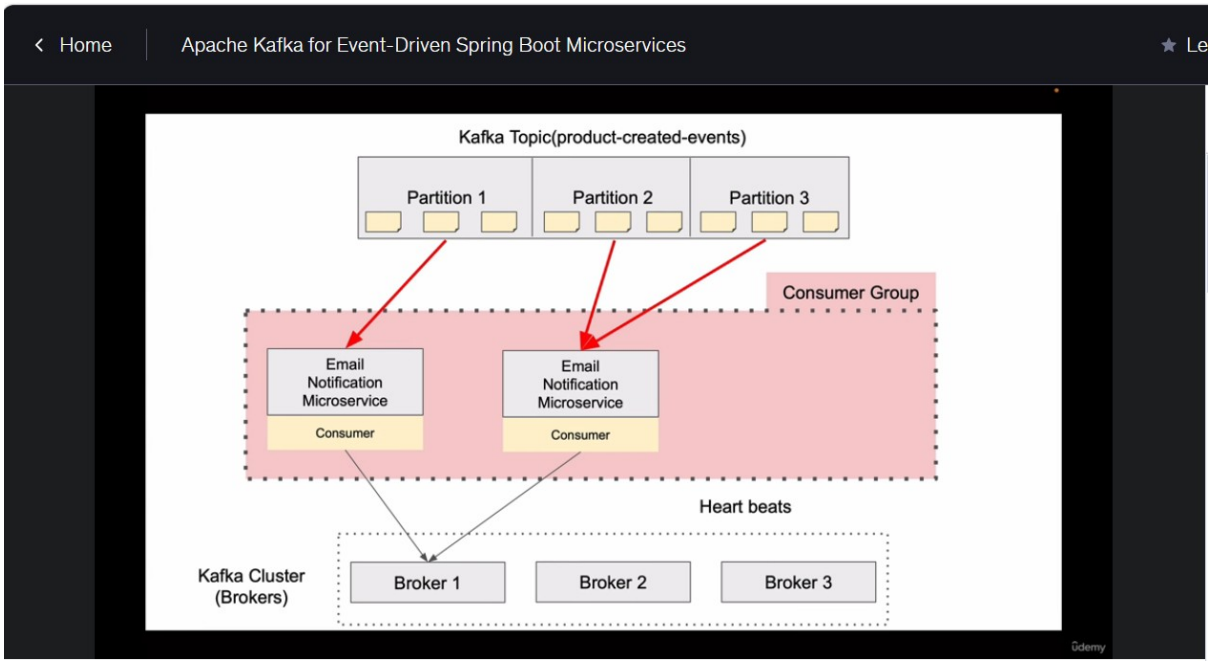


Consumer sends heart beats to broker session.timeout.ms (default its 3sec)

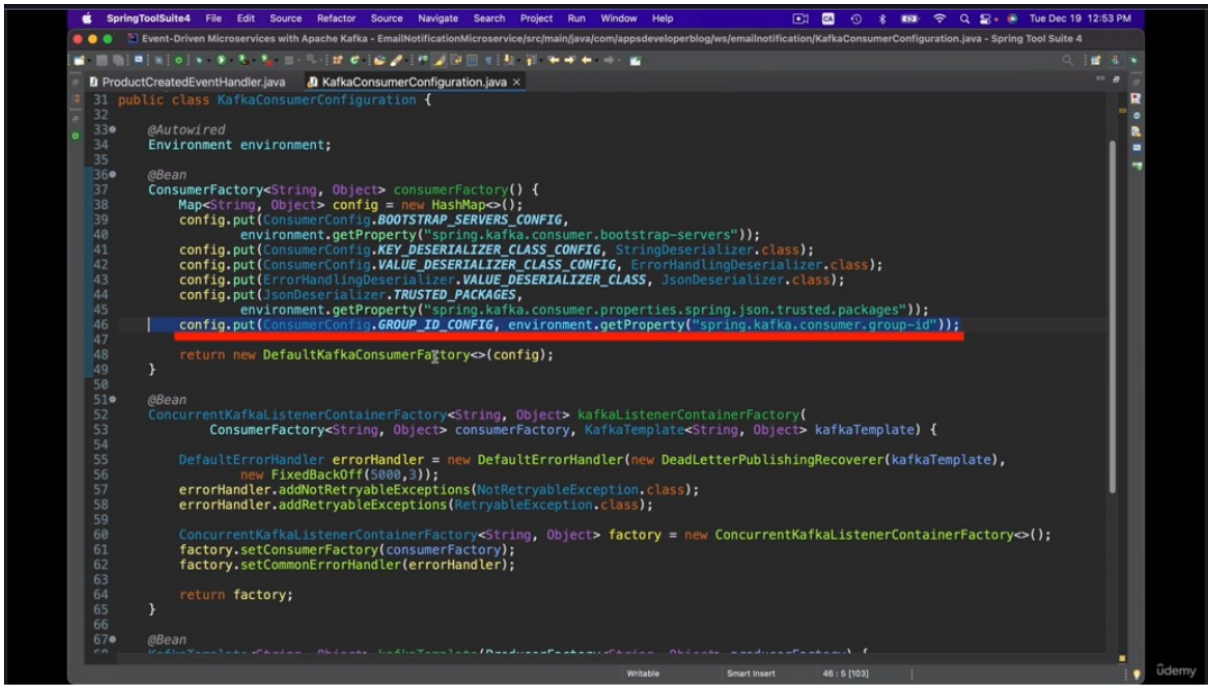


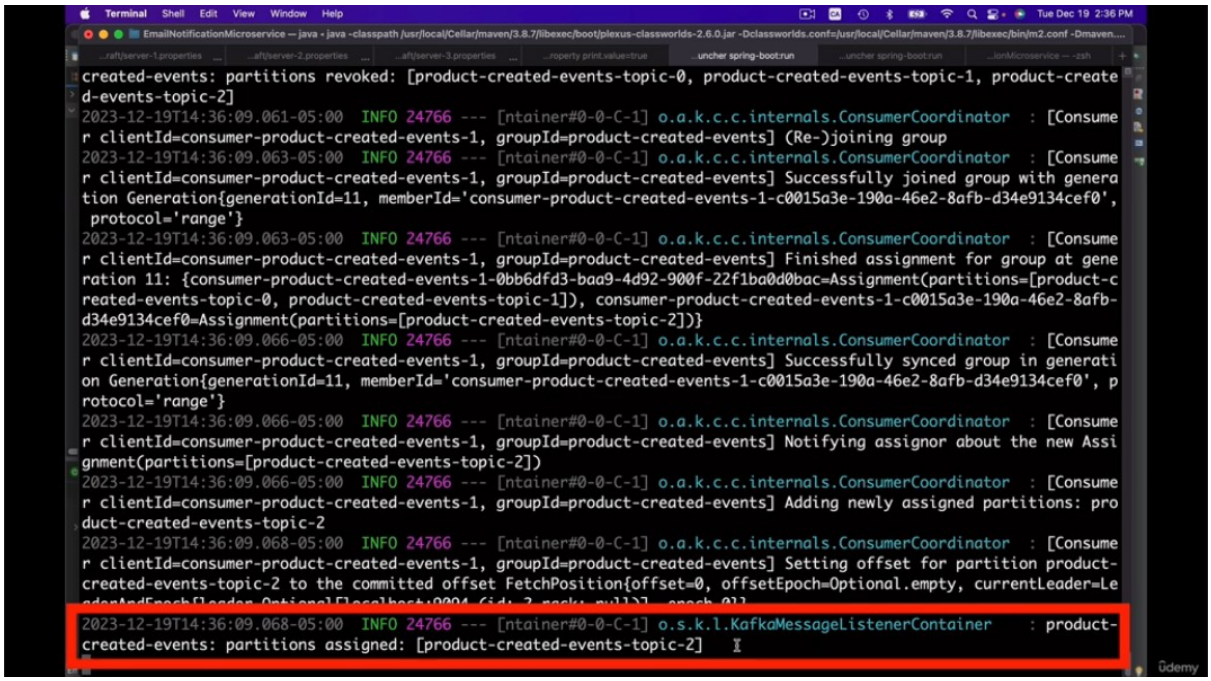
If no heartbeats are coming from the consumer, then the partition Is rebalanced to one of the healthier consumers





You can configure the group-id in either @KafkaListener or in mention it in the consumer-config.


SpringToolSuite4 File Edit Source Refactor Source Navigate Search Project Run Window Help
Tue Dec 19 12:53 PM
Event-Driven Microservices with Apache Kafka - EmailNotificationMicroservice/src/main/java/com/appsdeveloperblog/ws/emailnotification/KafkaConsumerConfiguration.java - Spring Tool Suite 4
ProductCreatedEventHandler.java
KafkaConsumerConfiguration.java
31 public class KafkaConsumerConfiguration {
32
33 @Autowired
34 Environment environment;
35
36 @Bean
37 ConsumerFactory<String, Object> consumerFactory() {
38 Map<String, Object> config = new HashMap<>();
39 config.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,
40 environment.getProperty("spring.kafka.consumer.bootstrap-servers"));
41 config.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class);
42 config.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, ErrorHandlingDeserializer.class);
43 config.put(ErrorHandlingDeserializer.VALUE_DESERIALIZER_CLASS, JsonDeserializer.class);
44 config.put(JsonDeserializer.TRUSTED_PACKAGES,
45 environment.getProperty("spring.kafka.consumer.properties.spring.json.trusted.packages"));
46 config.put(ConsumerConfig.GROUP_ID_CONFIG, environment.getProperty("spring.kafka.consumer.group-id"));
47
48 return new DefaultKafkaConsumerFactory<>(config);
49 }
50
51 @Bean
52 ConcurrentKafkaListenerContainerFactory<String, Object> kafkaListenerContainerFactory(
53 ConsumerFactory<String, Object> consumerFactory, KafkaTemplate<String, Object> kafkaTemplate) {
54
55 DefaultErrorHandler errorHandler = new DefaultErrorHandler(new DeadLetterPublishingRecoverer(kafkaTemplate),
56 new FixedBackOff(5000,3));
57 errorHandler.addNotRetryableExceptions(NotRetryableException.class);
58 errorHandler.addRetryableExceptions(RetryableException.class);
59
60 ConcurrentKafkaListenerContainerFactory<String, Object> factory = new ConcurrentKafkaListenerContainerFactory<>();
61 factory.setConsumerFactory(consumerFactory);
62 factory.setCommonErrorHandler(errorHandler);
63
64 return factory;
65 }
66
67 @Bean
Writable
Smart Insert
46 : 5 [103]
Udemy


Terminal Shell Edit View Window Help
Tue Dec 19 2:36 PM
EmailNotificationMicroservice — java • java -classpath /usr/local/Cellar/maven/3.8.7/libexec/boot/plexus-classworlds-2.6.0.jar -Dclassworlds.conf=/usr/local/Cellar/maven/3.8.7/libexec/bin/m2.conf -Dmaven....
created-events: partitions revoked: [product-created-events-topic-0, product-created-events-topic-1, product-created-events-topic-2]
2023-12-19T14:36:09.061-05:00 INFO 24766 --- [ntainer#0-0-C-1] o.a.k.c.c.internals.ConsumerCoordinator : [Consumer clientId=consumer-product-created-events-1, groupId=product-created-events] (Re-)joining group
2023-12-19T14:36:09.063-05:00 INFO 24766 --- [ntainer#0-0-C-1] o.a.k.c.c.internals.ConsumerCoordinator : [Consumer clientId=consumer-product-created-events-1, groupId=product-created-events] Successfully joined group with generation Generation{generationId=11, memberId='consumer-product-created-events-1-c0015a3e-190a-46e2-8afb-d34e9134cef0', protocol='range'}
2023-12-19T14:36:09.063-05:00 INFO 24766 --- [ntainer#0-0-C-1] o.a.k.c.c.internals.ConsumerCoordinator : [Consumer clientId=consumer-product-created-events-1, groupId=product-created-events] Finished assignment for group at generation 11: {consumer-product-created-events-1-0bb6dfd3-baa9-4d92-900f-22f1ba0d0bac=Assignment(partitions=[product-created-events-topic-0, product-created-events-topic-1]), consumer-product-created-events-1-c0015a3e-190a-46e2-8afb-d34e9134cef0=Assignment(partitions=[product-created-events-topic-2])}
2023-12-19T14:36:09.066-05:00 INFO 24766 --- [ntainer#0-0-C-1] o.a.k.c.c.internals.ConsumerCoordinator : [Consumer clientId=consumer-product-created-events-1, groupId=product-created-events] Successfully synced group in generation Generation{generationId=11, memberId='consumer-product-created-events-1-c0015a3e-190a-46e2-8afb-d34e9134cef0', protocol='range'}
2023-12-19T14:36:09.066-05:00 INFO 24766 --- [ntainer#0-0-C-1] o.a.k.c.c.internals.ConsumerCoordinator : [Consumer clientId=consumer-product-created-events-1, groupId=product-created-events] Notifying assignor about the new Assignment(partitions=[product-created-events-topic-2])
2023-12-19T14:36:09.066-05:00 INFO 24766 --- [ntainer#0-0-C-1] o.a.k.c.c.internals.ConsumerCoordinator : [Consumer clientId=consumer-product-created-events-1, groupId=product-created-events] Adding newly assigned partitions: product-created-events-topic-2
2023-12-19T14:36:09.068-05:00 INFO 24766 --- [ntainer#0-0-C-1] o.a.k.c.c.internals.ConsumerCoordinator : [Consumer clientId=consumer-product-created-events-1, groupId=product-created-events] Setting offset for partition product-created-events-topic-2 to the committed offset FetchPosition{offset=0, offsetEpoch=Optional.empty, currentLeader=Le
2023-12-19T14:36:09.068-05:00 INFO 24766 --- [ntainer#0-0-C-1] o.s.k.l.KafkaMessageListenerContainer : product-created-events: partitions assigned: [product-created-events-topic-2]
Udemy

What you know

What happens when a consumer in a Kafka Consumer Group shuts down?

What happens if the number of consumers in a Kafka Consumer Group exceeds the nu

What determines the number of partitions a consumer in a group can process in Kafka?

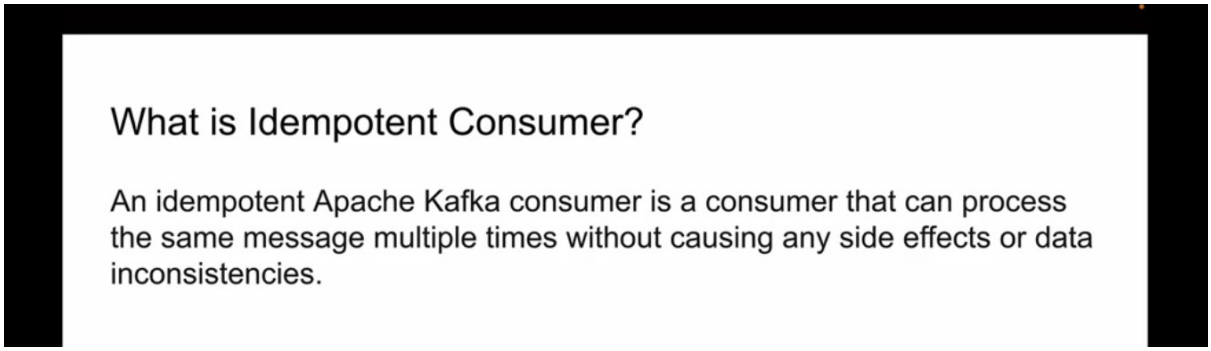
How does Apache Kafka assign partitions to consumers within a consumer group?

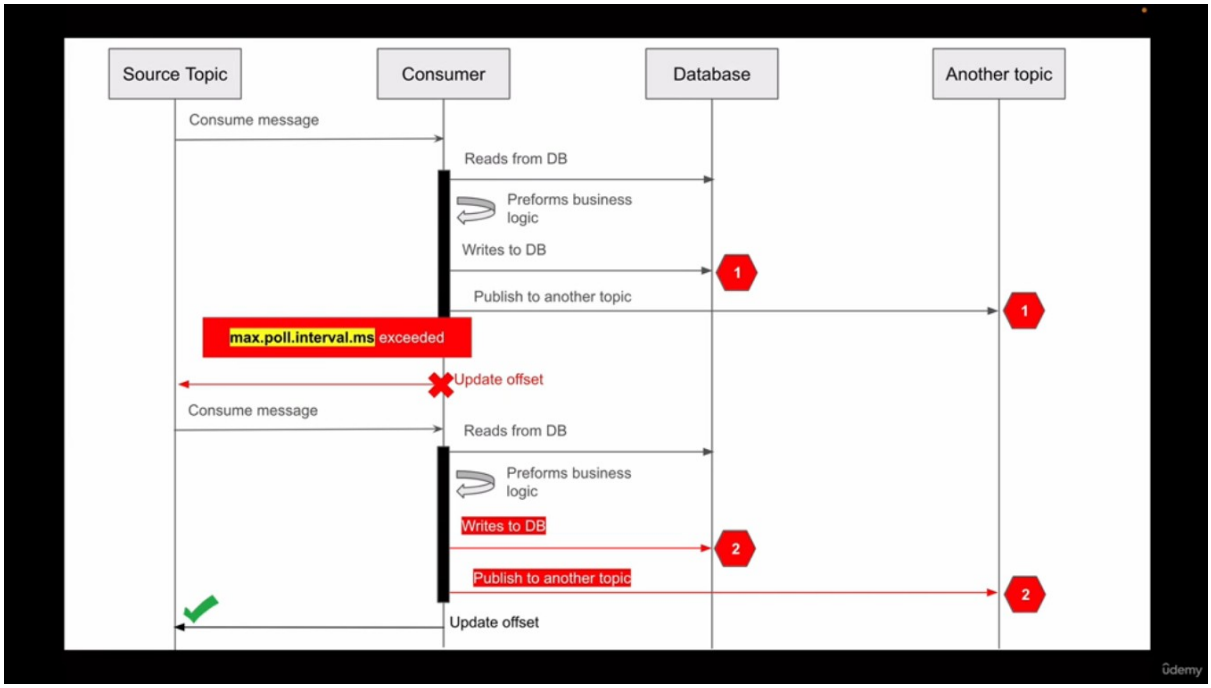
How does Kafka manage message distribution to consumers within a consumer group?

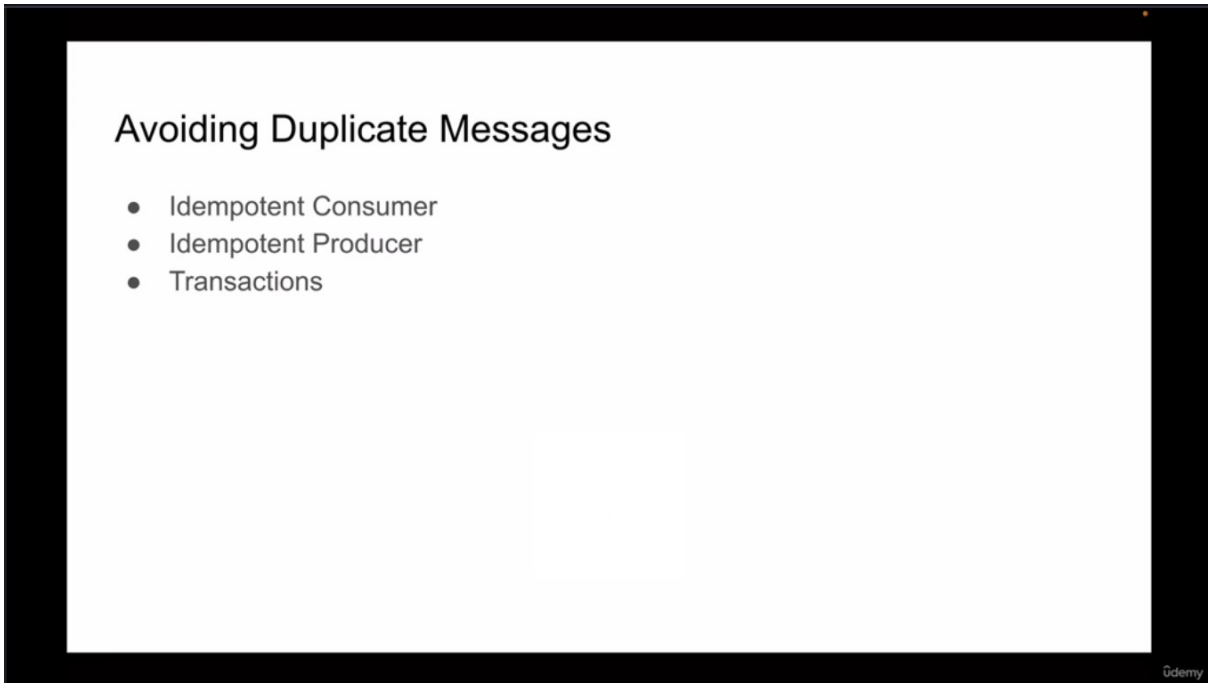
What happens when a new consumer joins a consumer group in Kafka?

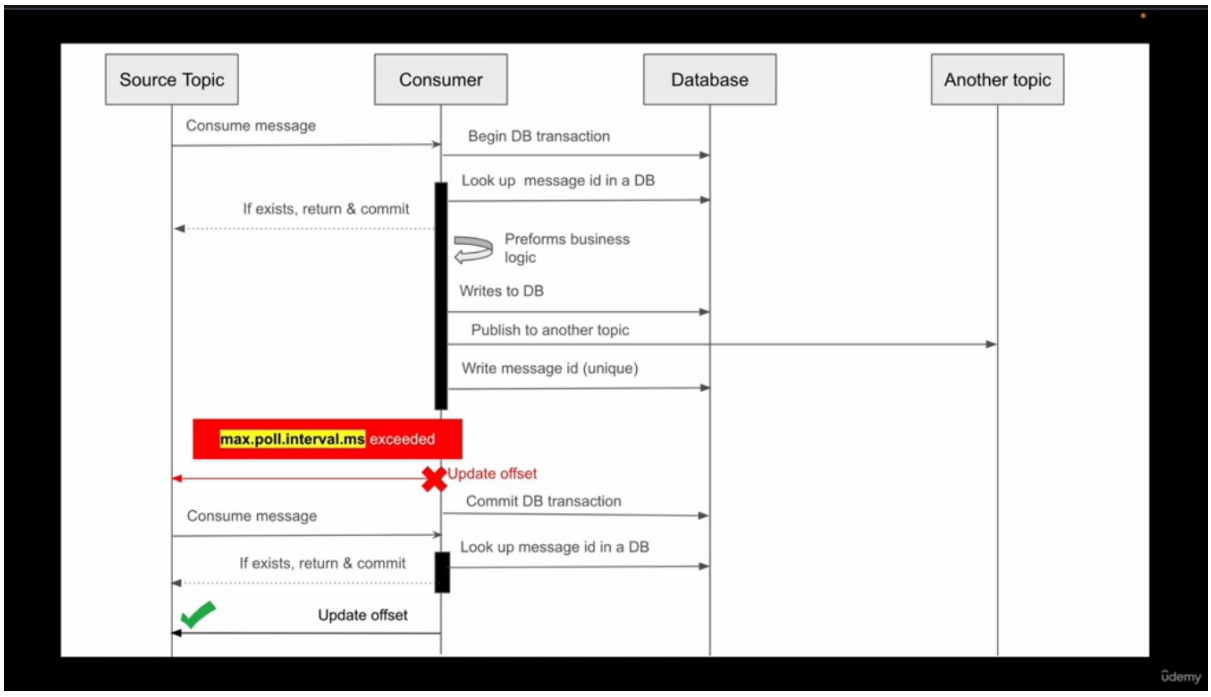
What is the primary function of Consumer Groups in Apache Kafka?

IDOMPETENT CONSUMER:





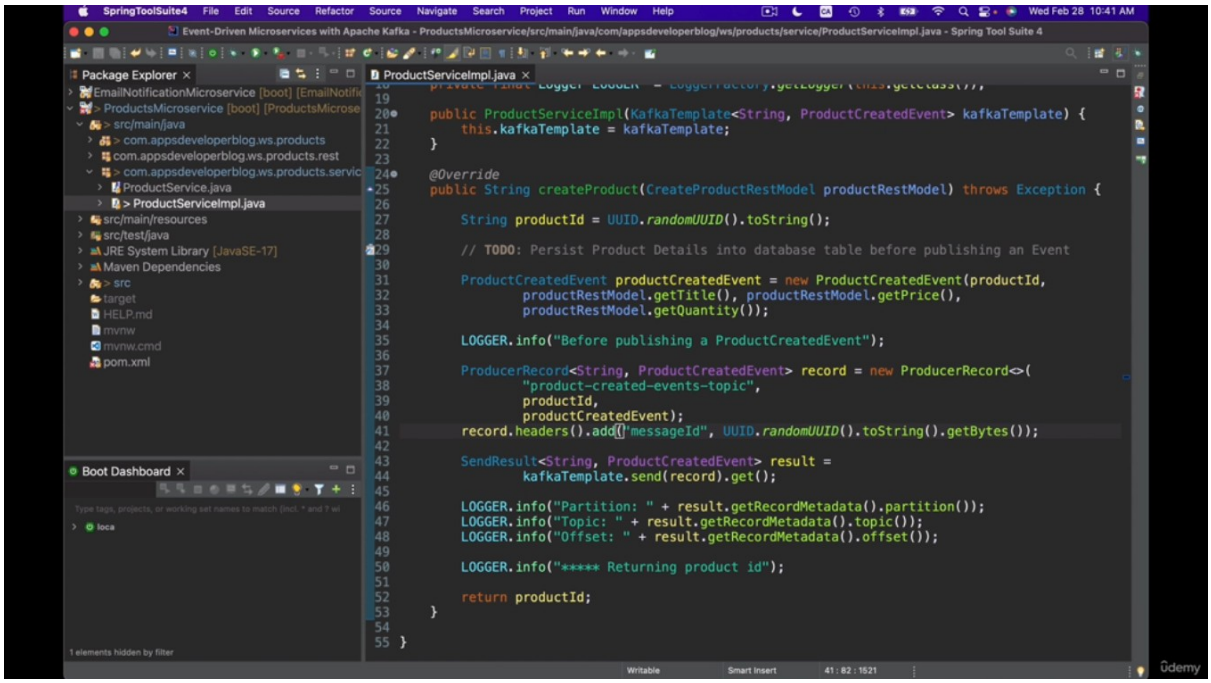


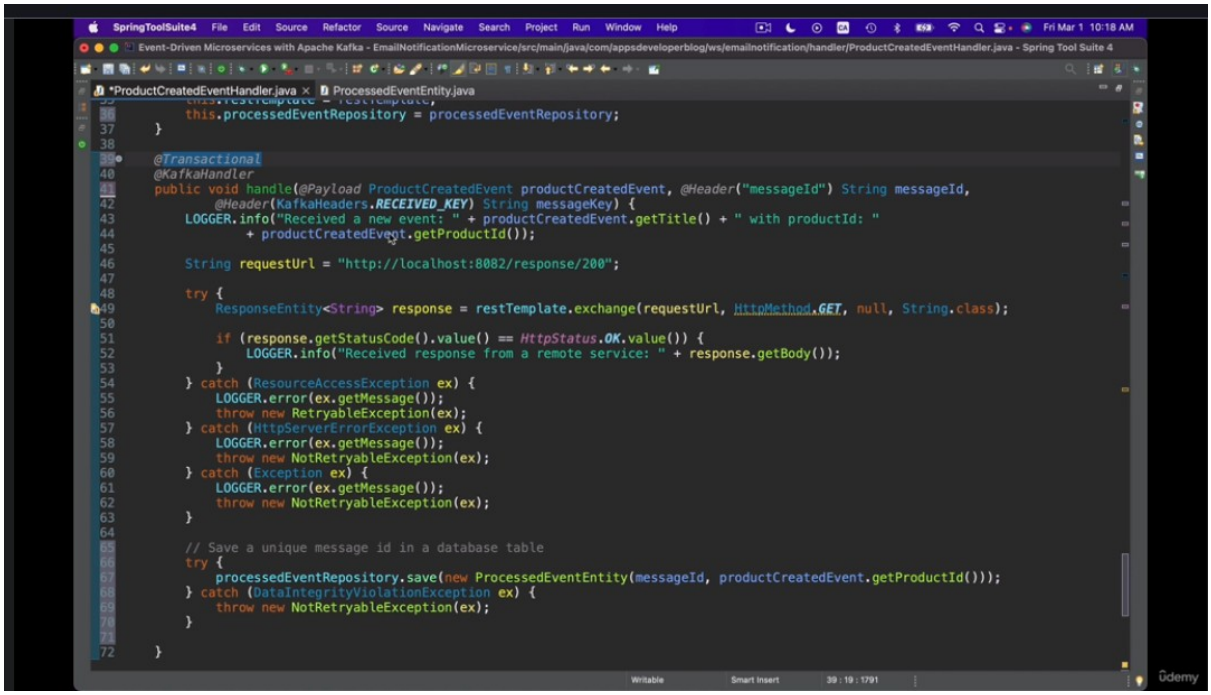


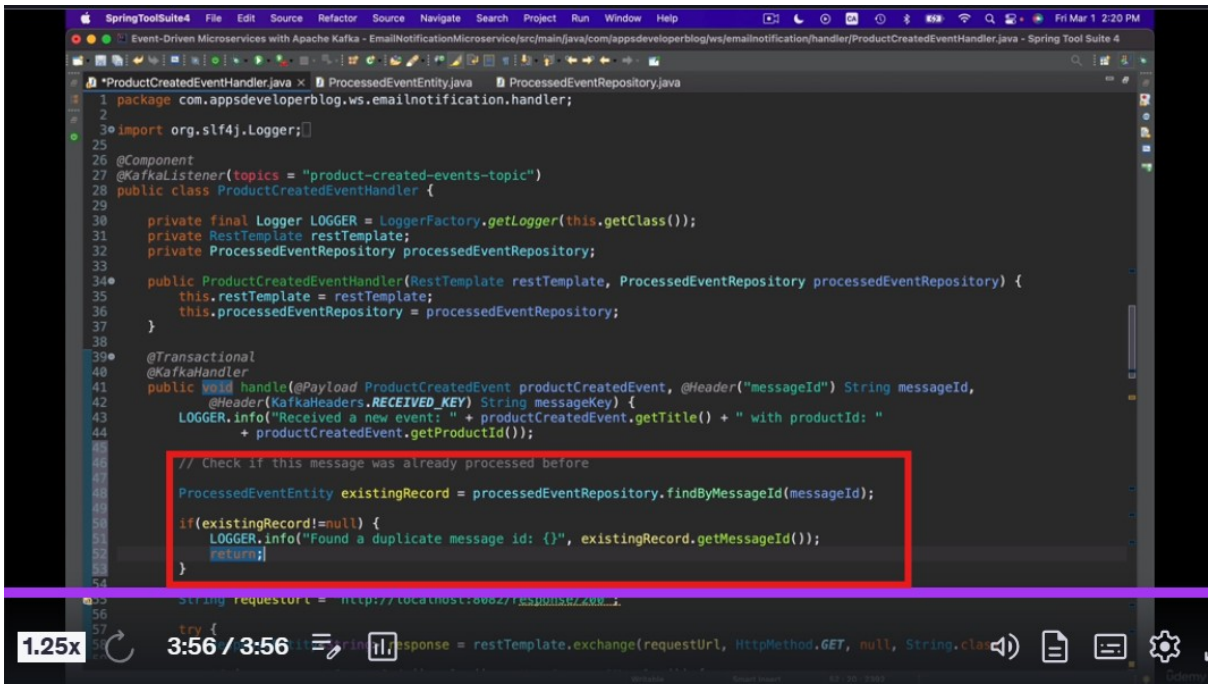
max.poll.interval.ms specifies the maximum delay allowed between two poll() calls. If a consumer does not call poll() within this interval, Kafka considers it unable to make progress, removes it from the consumer group, and triggers a rebalance. It prevents consumers from holding partitions indefinitely while being stuck processing records.

Session.timeout.ms = Is the consumer alive?

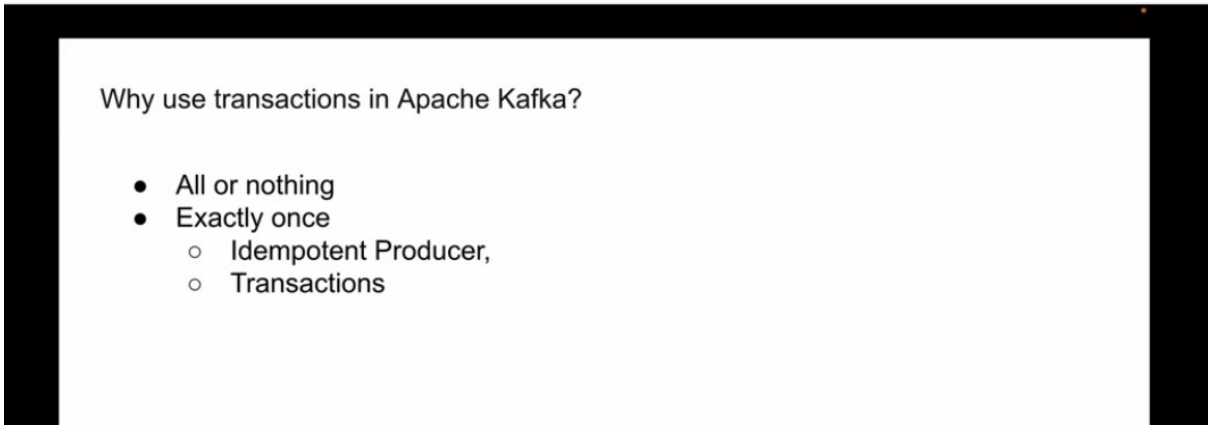
max.poll.interval.ms = Is the consumer still doing work and polling?



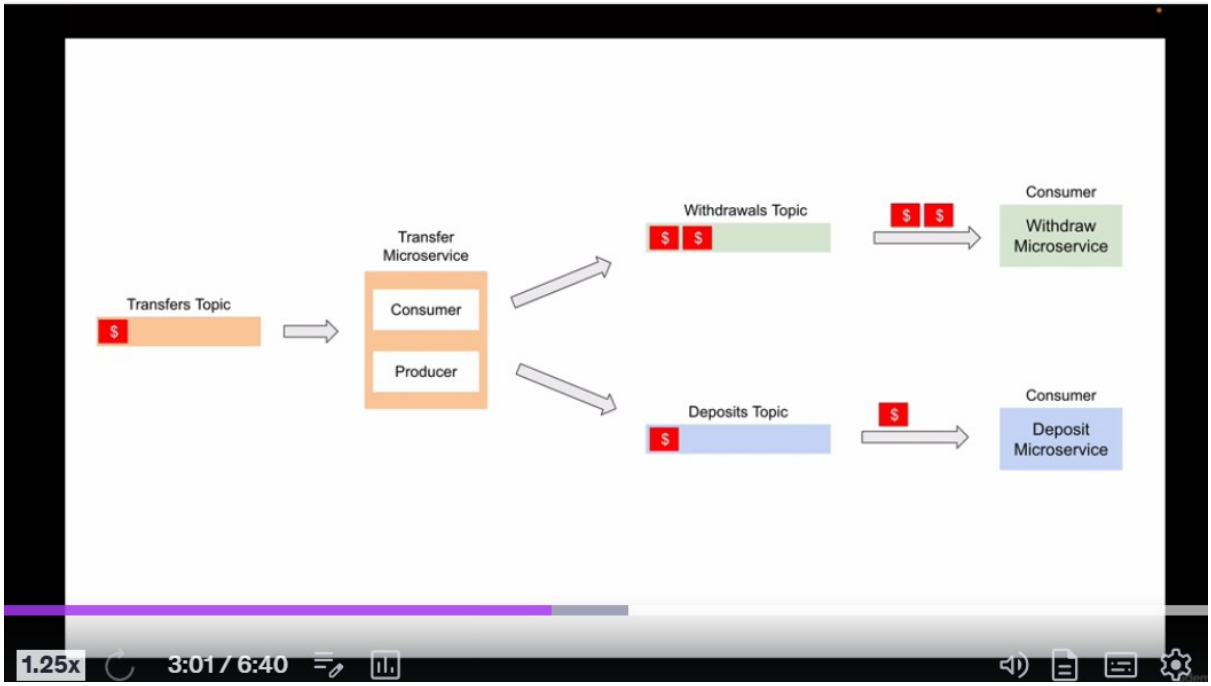




APACHE KAFKA TRANSACTIONS:



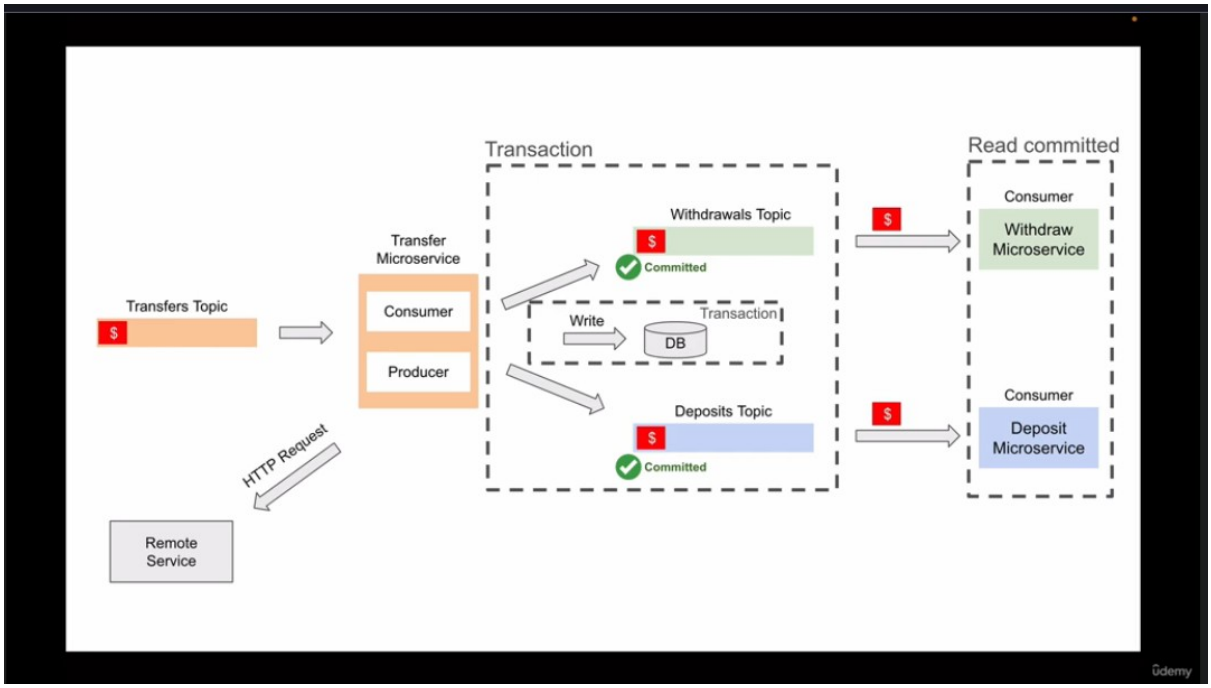
WITHOUT TRANSACTION:



WITH TRANSACTION:

In this we have producer that produces an event then, It is consumed by Transfer Microservice then the same microservice should produce the event to the withdraw topic and the deposit topic.
so we have something called read uncommitted and then committed then which helps use to make an environment which says either do all or

nothing




```
spring.kafka.producer.acks=all

spring.kafka.producer.properties.delivery.timeout.ms=120000
spring.kafka.producer.properties.linger.ms=0
spring.kafka.producer.properties.request.timeout.ms=30000

spring.kafka.producer.properties.max.in.flight.requests.per.connection=5
spring.kafka.producer.properties.enable.idempotence=true

withdraw-money-topic=withdraw-money-topic
deposit-money-topic=deposit-money-topic

spring.kafka.producer.transaction-id-prefix=transfer-service
```

Full Example

YAML

```yaml
spring:
  kafka:
    bootstrap-servers: localhost:9092

    producer:
      acks: all
      retries: 5
      transaction-id-prefix: tx-

      properties:
        enable.idempotence: true

    consumer:
      group-id: order-group
      auto-offset-reset: earliest
      enable-auto-commit: false
```

Show more lines

Listener Side

Java

```java
@KafkaListener(topics = "orders")
@Transactional
public void process(OrderEvent event) {

    kafkaTemplate.send(
        "inventory-topic",
        inventoryEvent
    );

    kafkaTemplate.send(
        "payment-topic",
        paymentEvent
    );
}
```

Spring opens a Kafka transaction.

The key properties are:

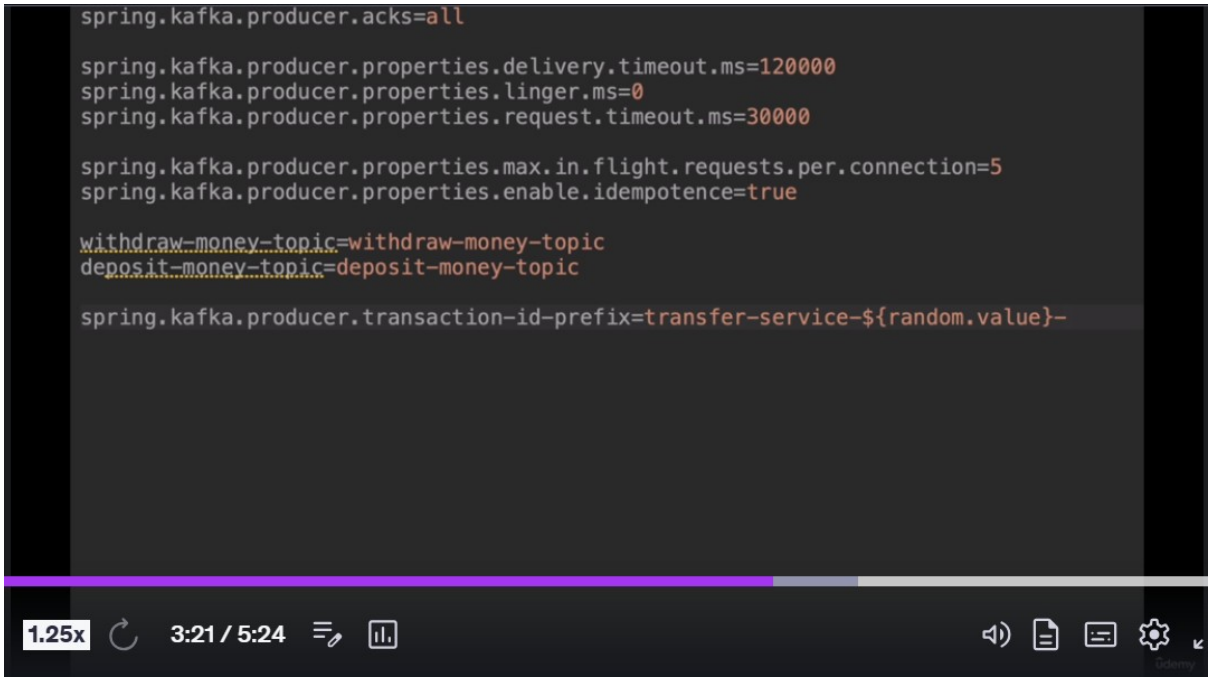
```
transaction-id-prefix
enable.idempotence=true
acks=all
enable-auto-commit=false
isolation.level=read_committed
```

Consumer Isolation Level

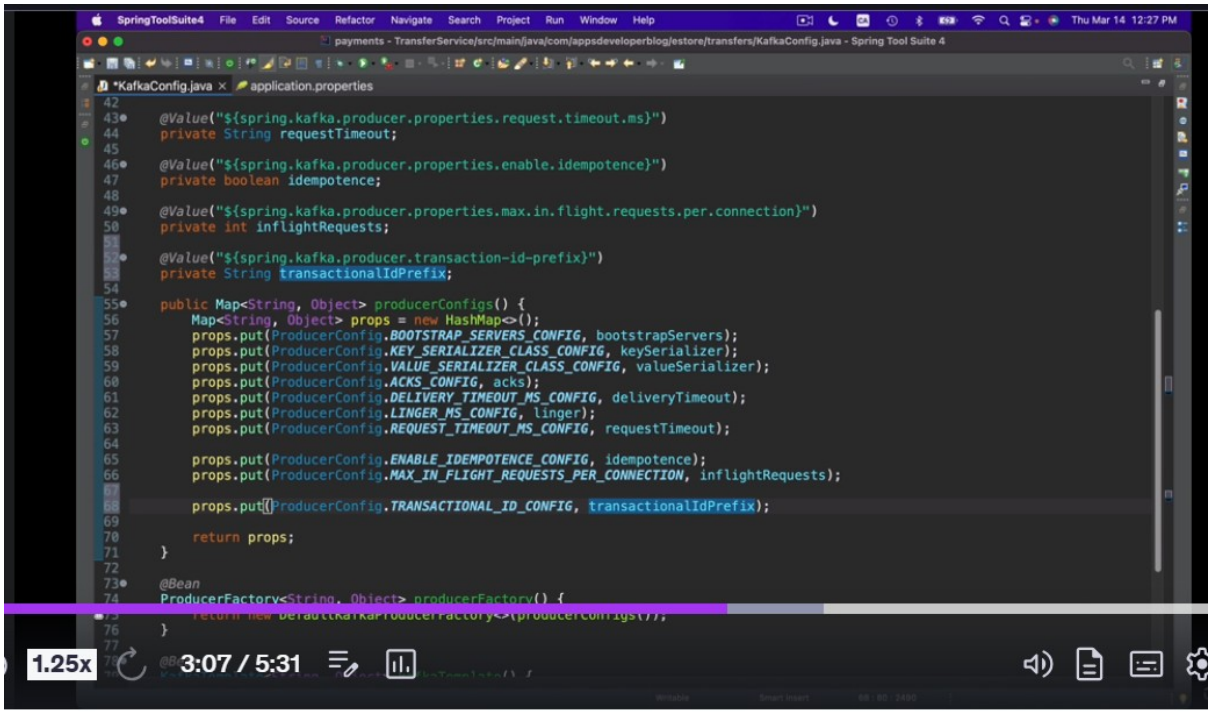
Another important property many people forget:

```yaml
spring:
  kafka:
    consumer:
      properties:
        isolation.level: read_committed
```

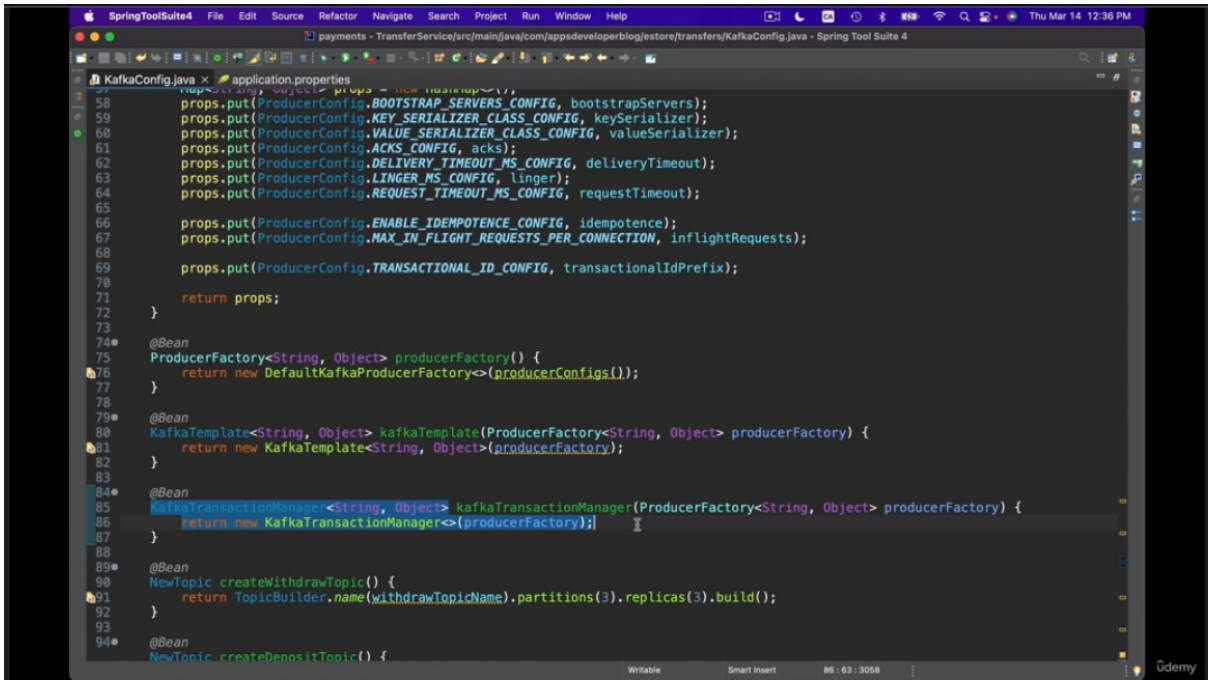
If you run multiple instances of the service running use this ${random.value}

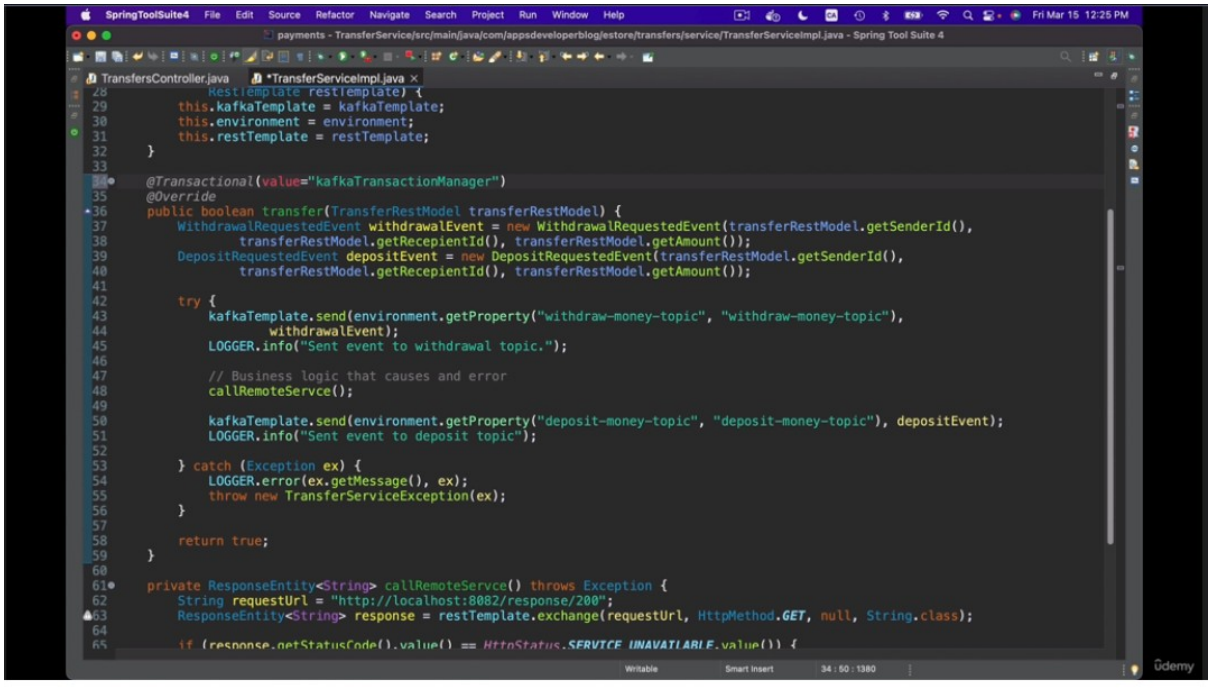


Use java bean config for kafka transactions



Create a bean for the Kafka Transactional Manager





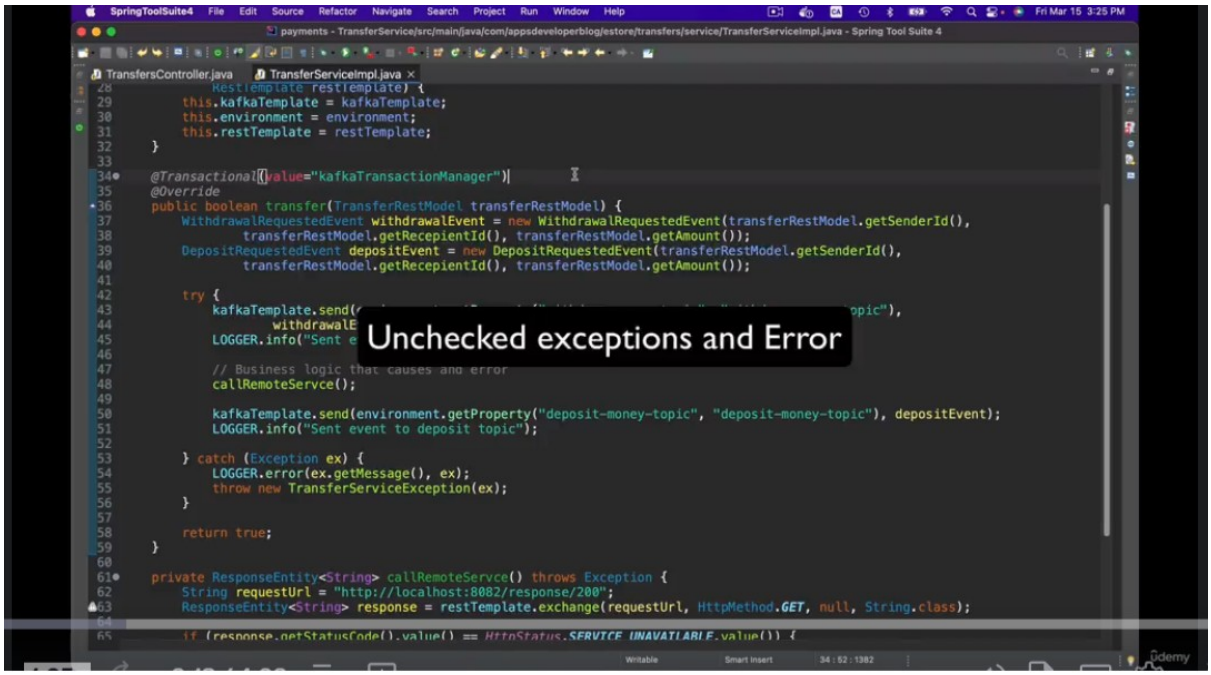
Production Recommendation

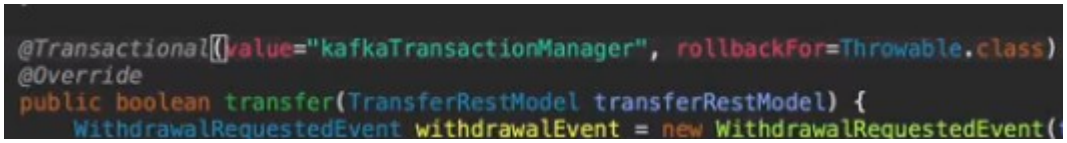
Plain Text

```
Database + Outbox Table + Kafka Publisher
```

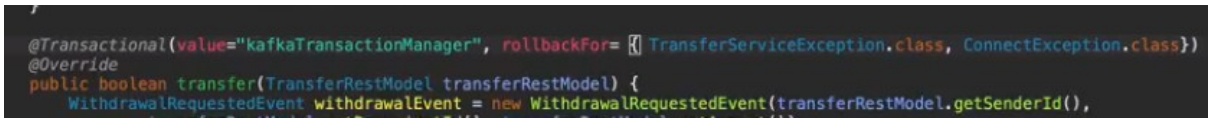
This is the pattern you'll most commonly see in real Spring Boot microservices.

This is as usual transactional annotation behaves only for unchecked and erro

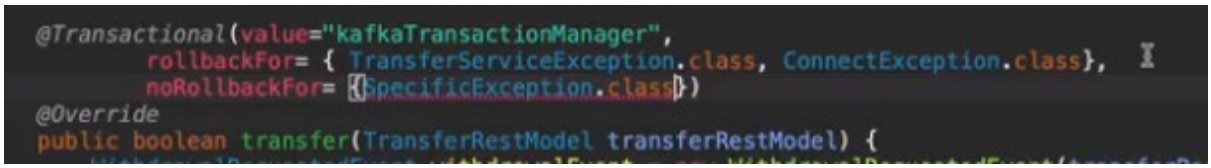




Can provide more than one exception by { , , … }



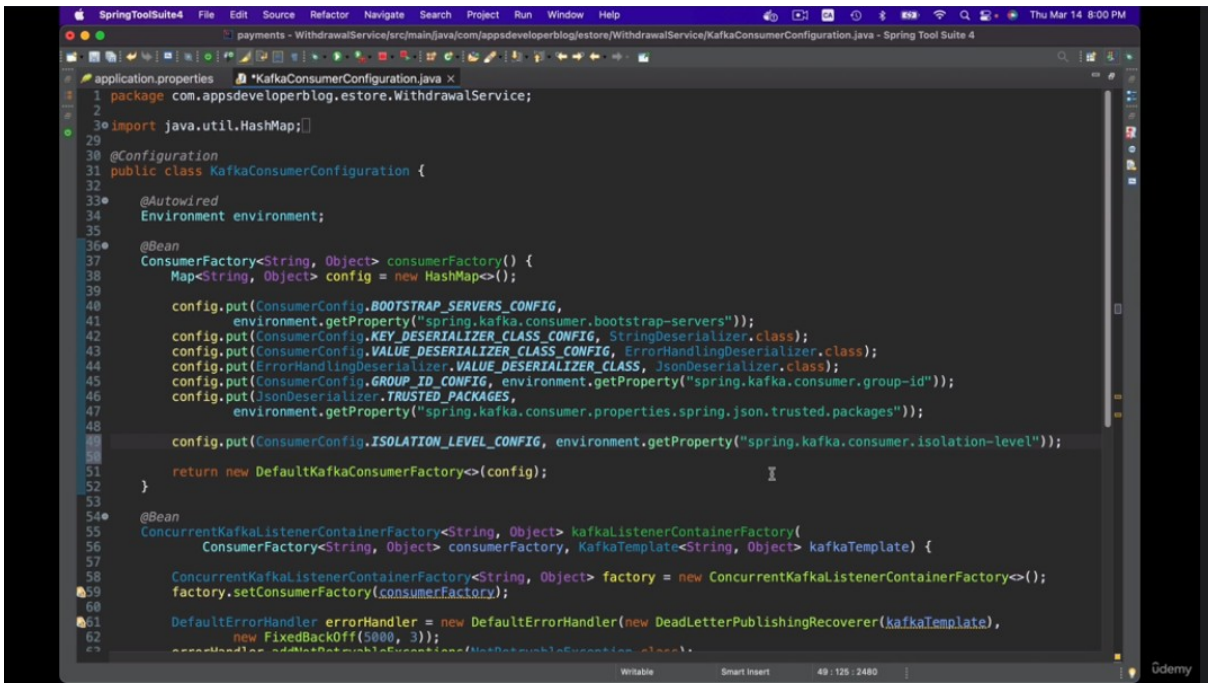
We also have no rollback for as well forn the exception we don't need to rollback



Default value In the consumer service is read uncommitted, but if we need the kafka transactions to work properly then we should set the value as read_committed



Bean config for the consumer isolation level….



KAFKA TEMPLATE -> executeInTransaction

```
boolean returnValue = kafkaTemplate.executeInTransaction(t -> {

        t.send("withdraw-money-topic", withdrawalEvent);

          throws Exception
        callRemoteServce();

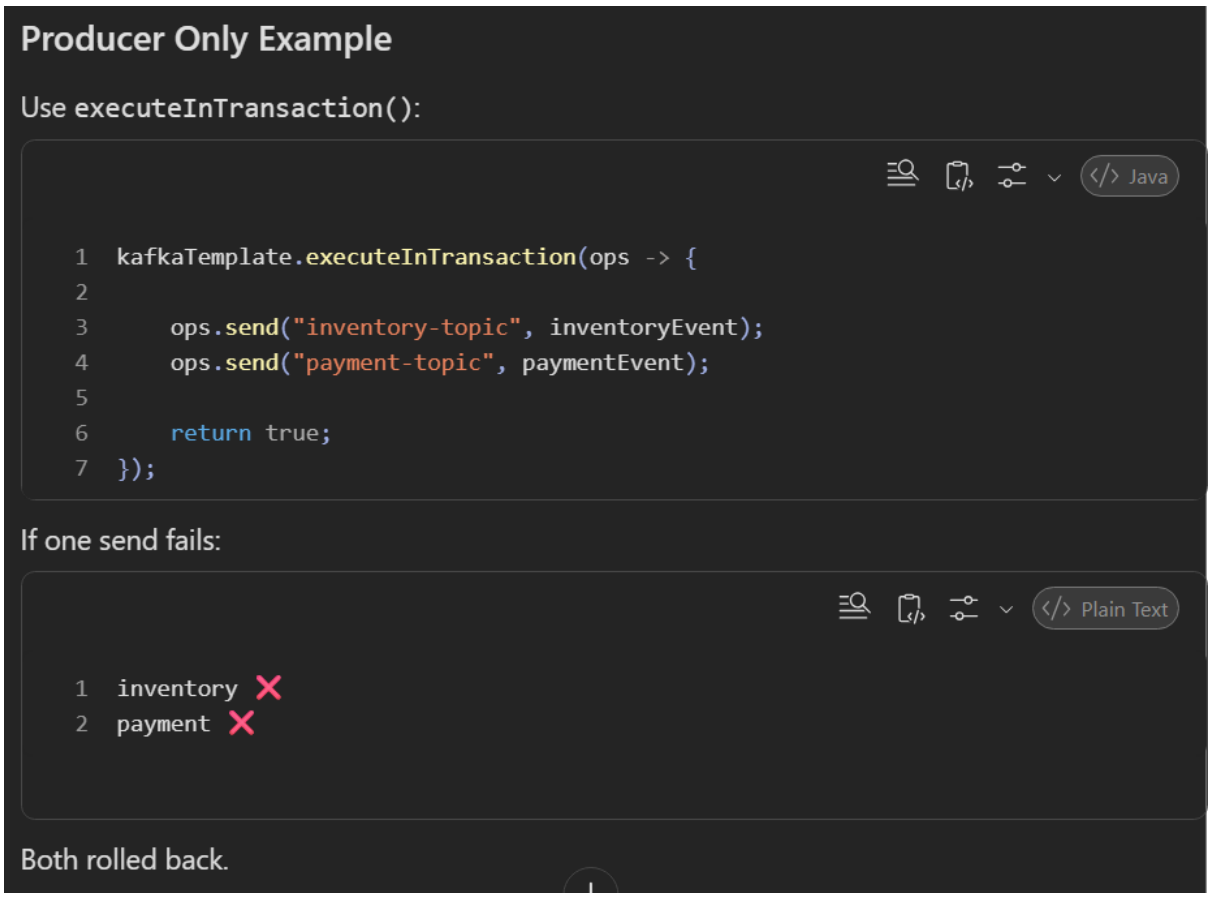
        t.send("deposit-money-topic", depositEvent);

        return true;

});
```

Feature	executeInTransaction()	@Transactional("kafkaTransactionManager")
Purpose	Transaction around a block of Kafka sends	Transaction around an entire method
Who starts transaction?	KafkaTemplate	Spring Transaction Manager
Producer sends atomic?	✅ Yes	✅ Yes
Works with @KafkaListener?	❌ Not directly	✅ Yes
Can include consumer offset commits?	❌ No	✅ Yes
Consume → Process	❌ Not ideal	✅ Main use case

→ Produce flow		
Method rollback support	**❌ Only code inside callback**	**✅ Whole method**
Requires transaction-id-prefix?	**✅ Yes**	**✅ Yes**
Requires KafkaTransactionManager bean?	**❌ No explicit usage**	**✅ Yes (Spring uses it)**
Multiple topic writes atomically	**✅ Yes**	**✅ Yes**
Best for	**Producer-only use cases**	**Consumer + Producer workflows**



Consumer → Producer Example

Use:

```java
@KafkaListener(topics = "orders")
@Transactional("kafkaTransactionManager")
public void process(OrderEvent event) {

    kafkaTemplate.send("inventory-topic", inventoryEvent);

    kafkaTemplate.send("payment-topic", paymentEvent);
}
```

Now Spring handles:

```
Consume record
    ↓
Produce records
    ↓
Commit offsets
```

all in one Kafka transaction.

Scope of the Transaction

```
@Transactional("kafkaTransactionManager")
public void transfer(TransferRestModel transferRestModel)  {

    ...

            kafkaTemplate.send("withdraw-money-topic", withdrawalEvent);

    thow new Exception("Force error");

            kafkaTemplate.send("deposit-money-topic", depositEvent);

    ...

}
```

Udemy

Scope of the Transaction

```
public void transfer(TransferRestModel transferRestModel)  {

        boolean returnValue = kafkaTemplate.executeInTransaction(t -> {

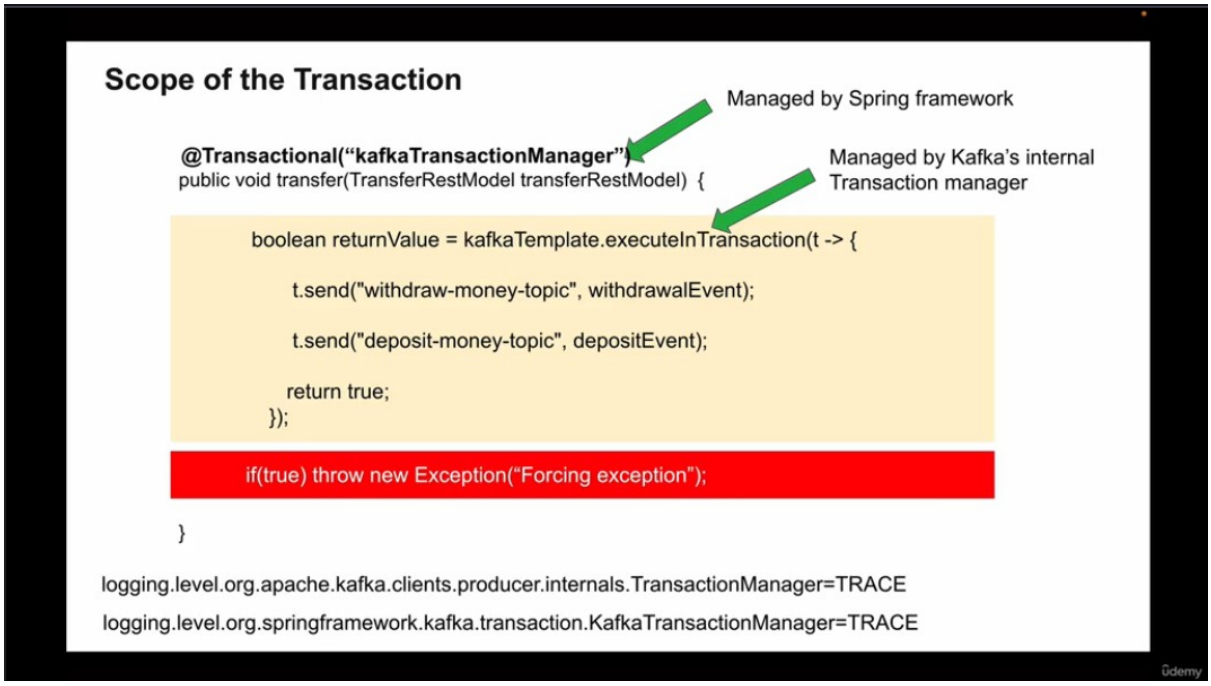
            t.send("withdraw-money-topic", withdrawalEvent);

            t.send("deposit-money-topic", depositEvent);

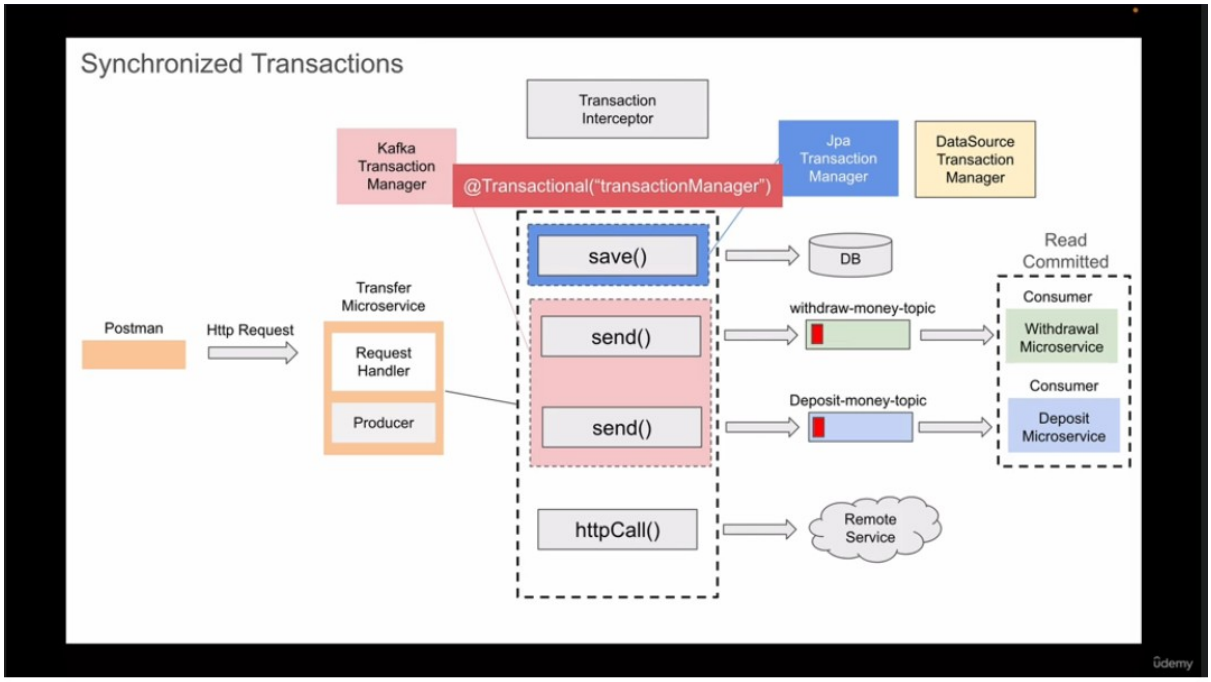
           return true;
         });

        if(true) throw new Exception("Forcing exception");

}
```



KAFKA & DATABASE TRANSACTIONS




```
Begin DB Transaction
      ↓
Save Order
      ↓
DB Commit
      ↓
AFTER_COMMIT callback
      ↓
Publish Kafka Event
```

Example:

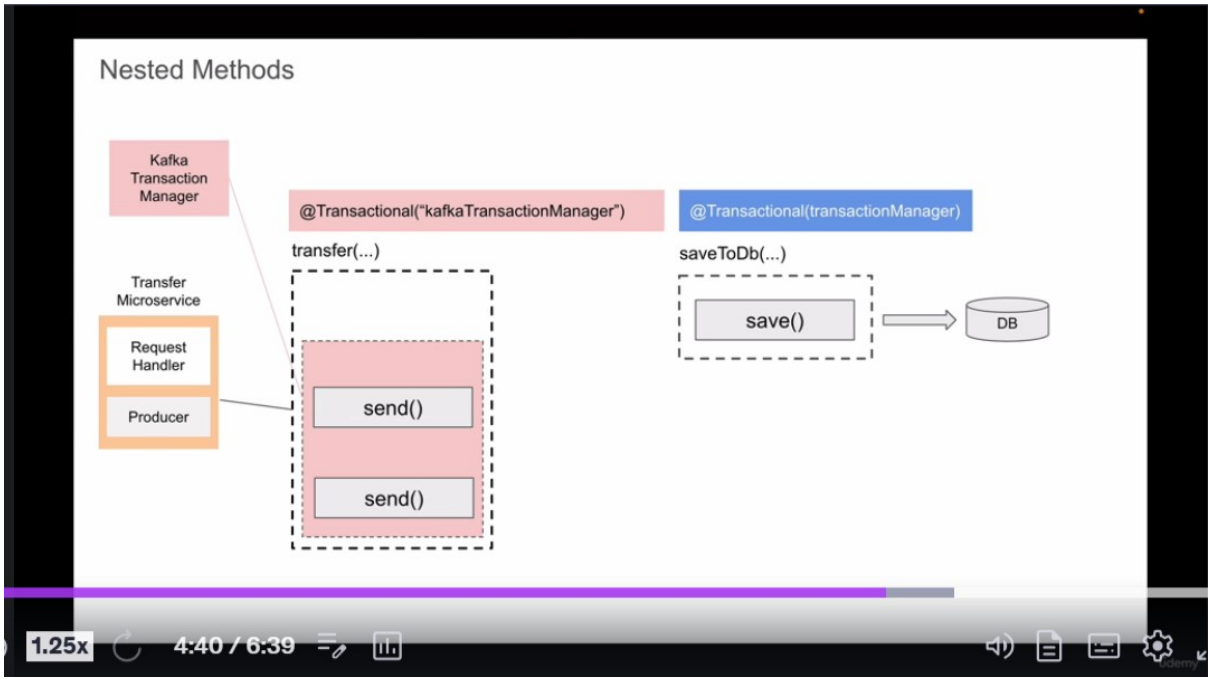
```java
@Transactional
public void createOrder() {

    orderRepository.save(order);

    TransactionSynchronizationManager
        .registerSynchronization(
            new TransactionSynchronization() {

                @Override
                public void afterCommit() {

                    kafkaTemplate.send(...);
                }
            });
}
```



Scenario 3: Different Methods

```java
@Service
public class OrderService {

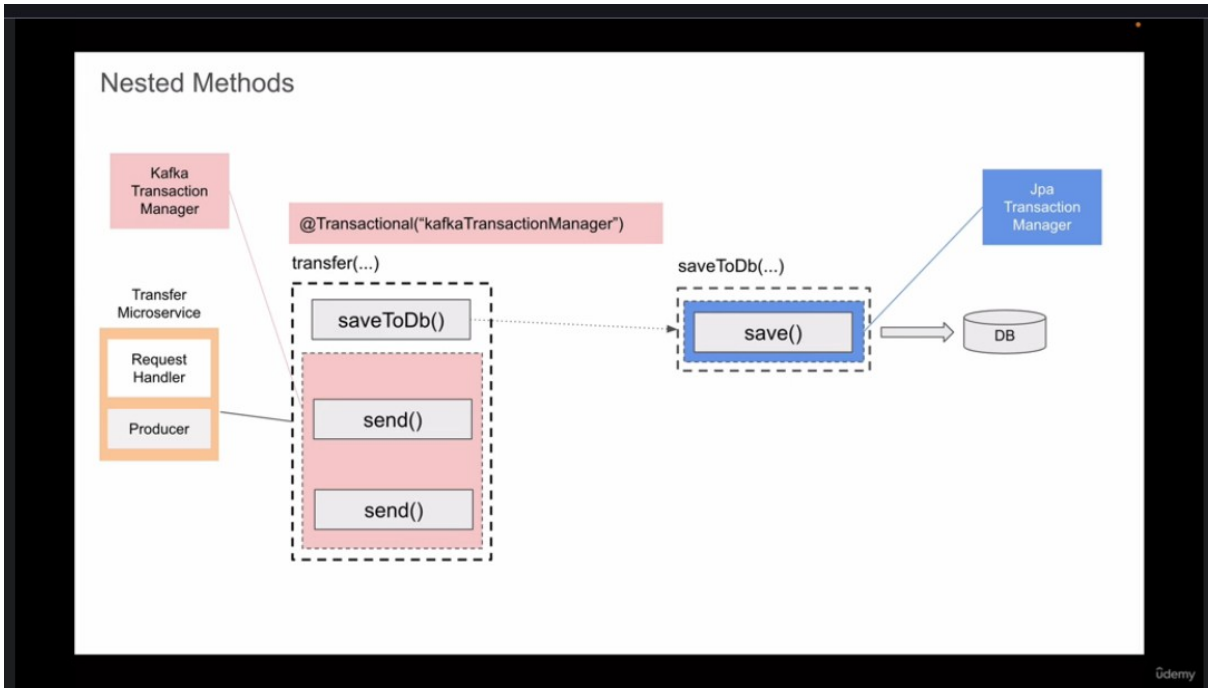
    @Transactional
    public void saveOrder() {
        orderRepository.save(...);
    }
}
```

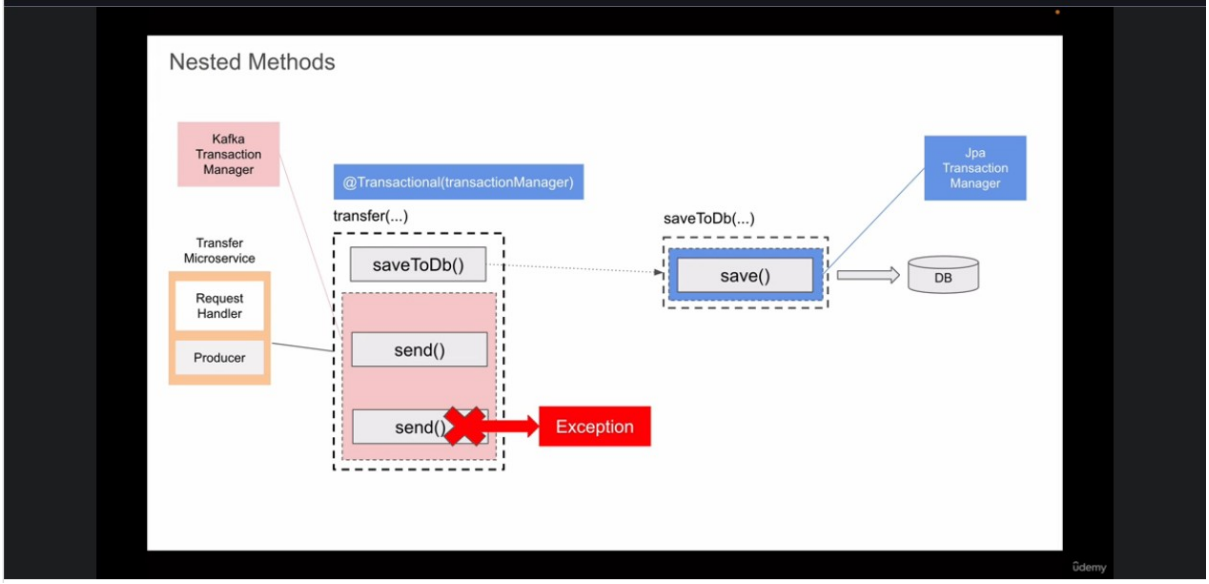
and

```java
@Service
public class EventService {

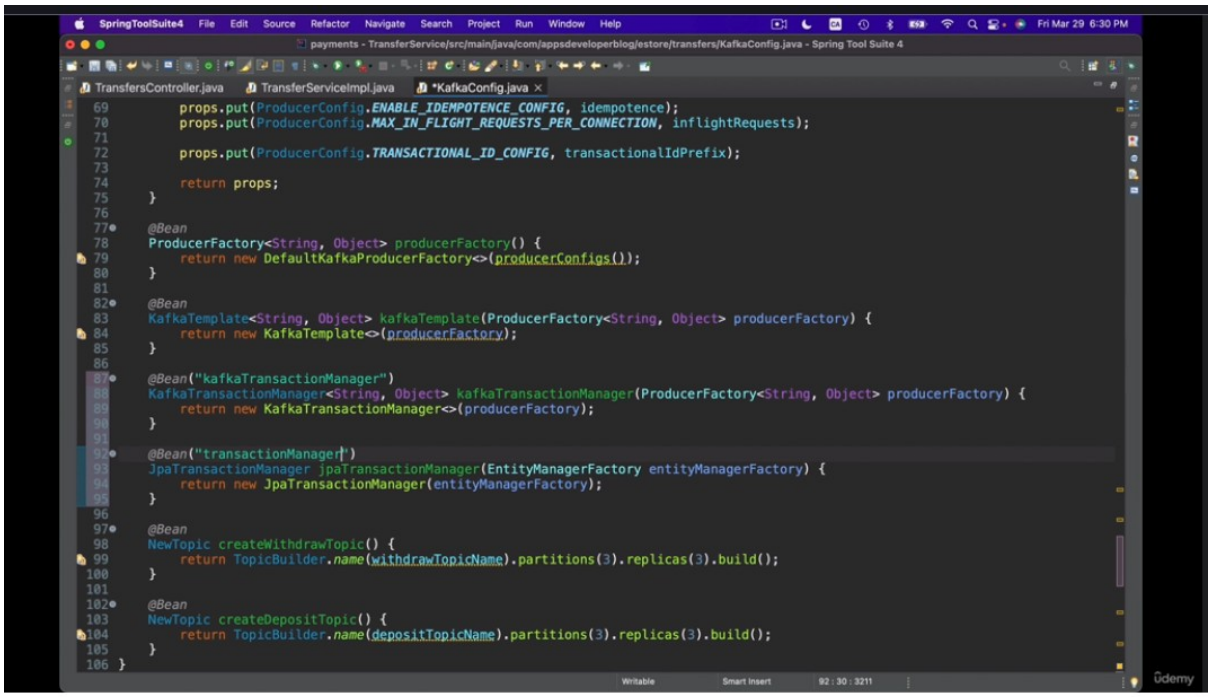
    @Transactional("kafkaTransactionManager")
    public void publish() {
        kafkaTemplate.send(...);
    }
}
```

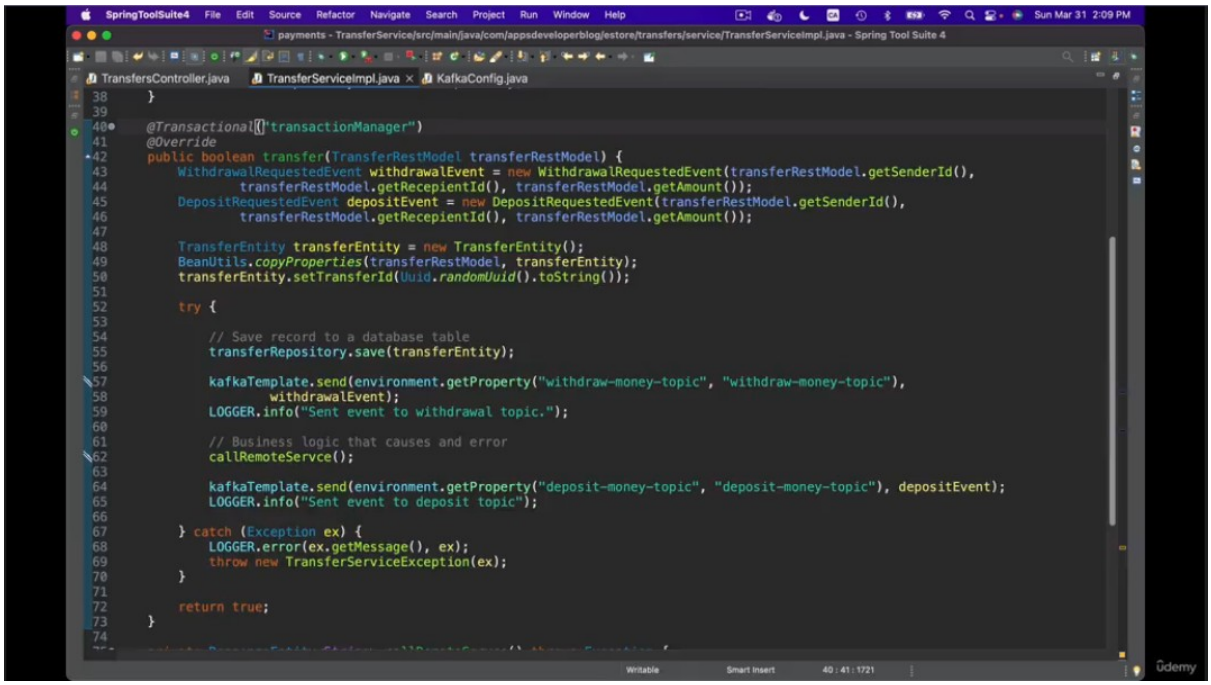
Now each method explicitly uses its own tra[illegible]tion manager.


Nested Methods
Kafka Transaction Manager
@Transactional("kafkaTransactionManager")
transfer(...)
saveToDb(...)
Jpa Transaction Manager
Transfer Microservice
saveToDb()
save()
DB
Request Handler
Producer
send()
send()
Udemy


Nested Methods
Kafka Transaction Manager
@Transactional(transactionManager)
transfer(...)
saveToDb(...)
Jpa Transaction Manager
Transfer Microservice
saveToDb()
save()
DB
Request Handler
Producer
send()
send()
Exception
Udemy

Spring by default uses the default transaction manager, which is typically JpaTransactionManager. To use Kafka transactions explicitly, I would use @Transactional("kafkaTransactionManager"). You can or don't even need to create the bean





Option 1: Use JPA Transaction Manager (more common)

```java
@Transactional
public void createOrder() {

    orderRepository.save(order);

    kafkaTemplate.send("orders", event);
}
```

Here:

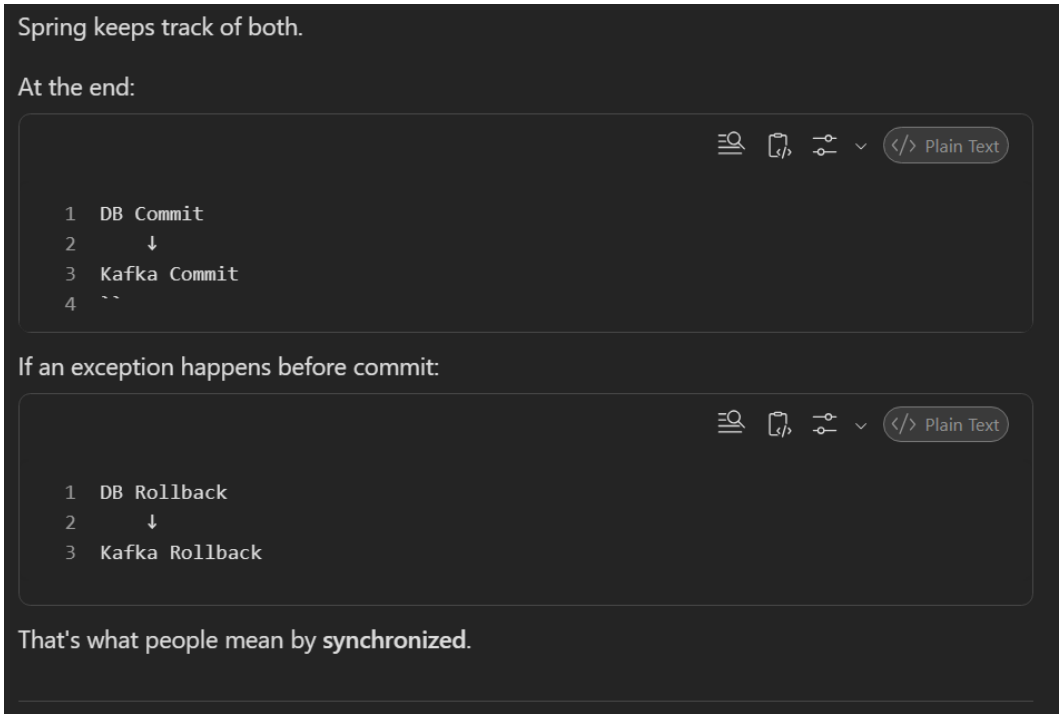
```
JpaTransactionManager
        |
        +--> DB transaction
        |
        +--> Kafka transaction synchronized
```

Requirements:

```yaml
spring:
  kafka:
    producer:
      transaction-id-prefix: tx-
```

because your primary transaction is the DB transaction.

Spring can synchronize Kafka producer operations with the existing transaction.



When you first learn Kafka, it looks like:

Producer ---> Kafka ---> Consumer

Simple.

Then you discover:

What if producer retries?

→ Idempotence

Then:

What if producer writes to multiple topics?

→ Transactions

Then:

What if consumer reads before commit?

→ read_committed

Then:

What if DB and Kafka both need to succeed?

→ Outbox Pattern

Then:

What if scheduler publishes twice?

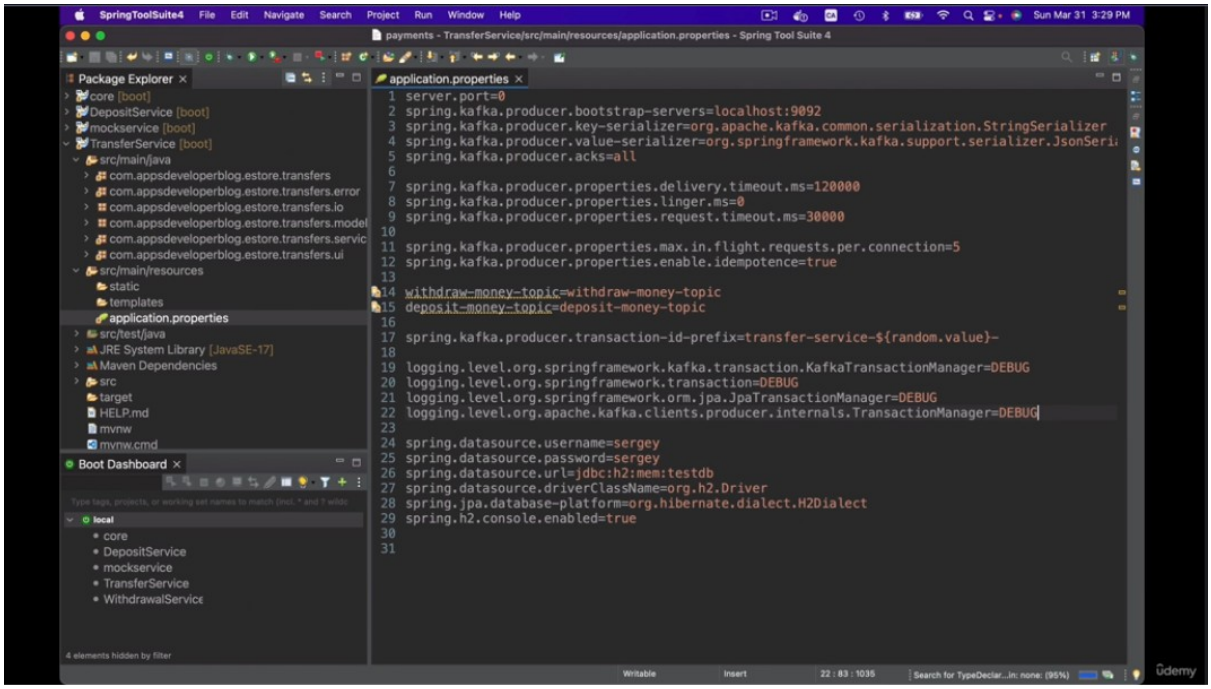
→ Idempotent Consumers

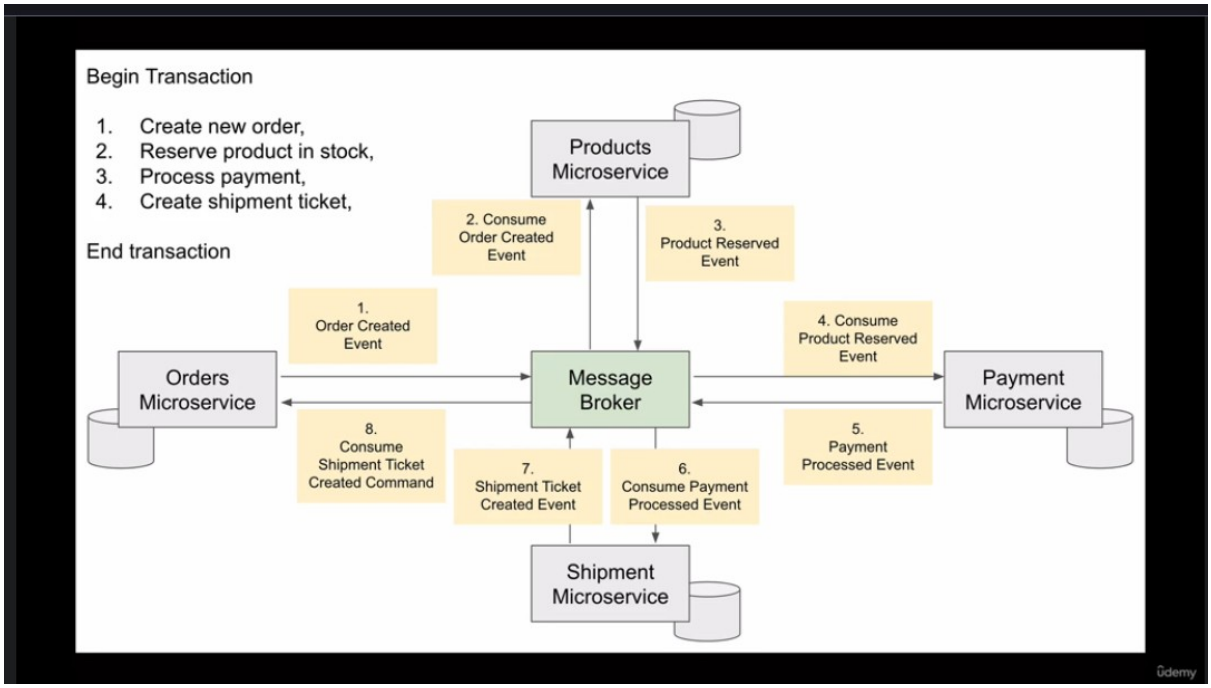
Then:

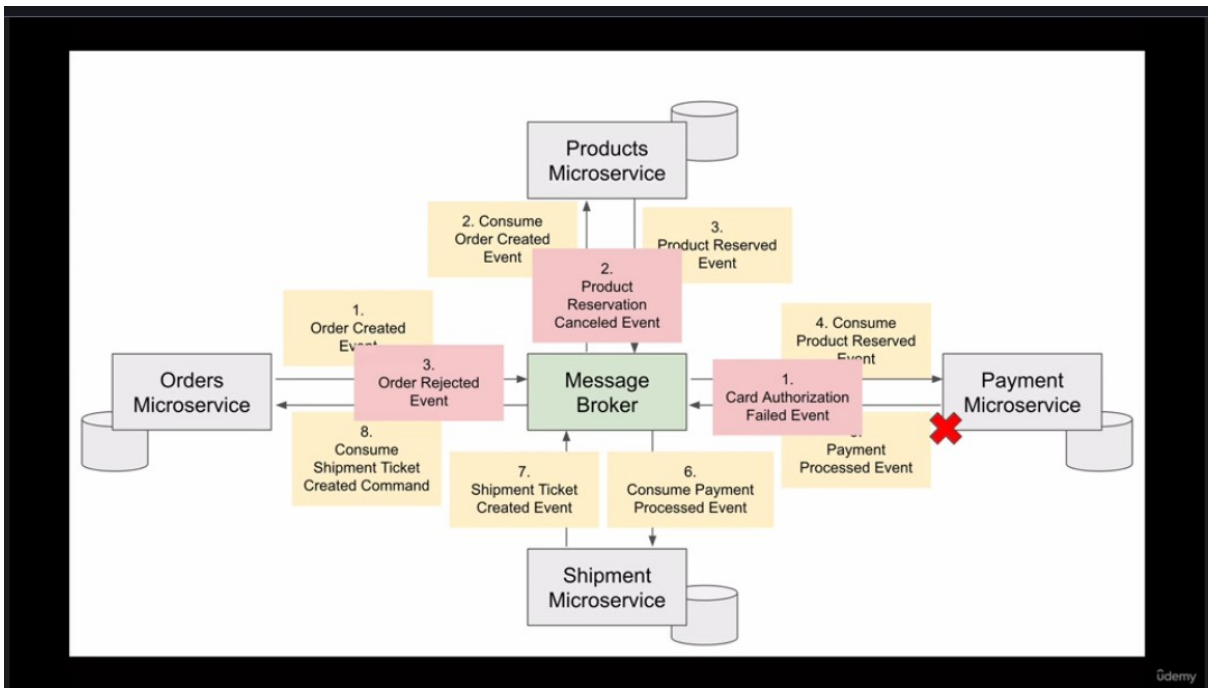
What if scheduler crashes after publish but before status update?

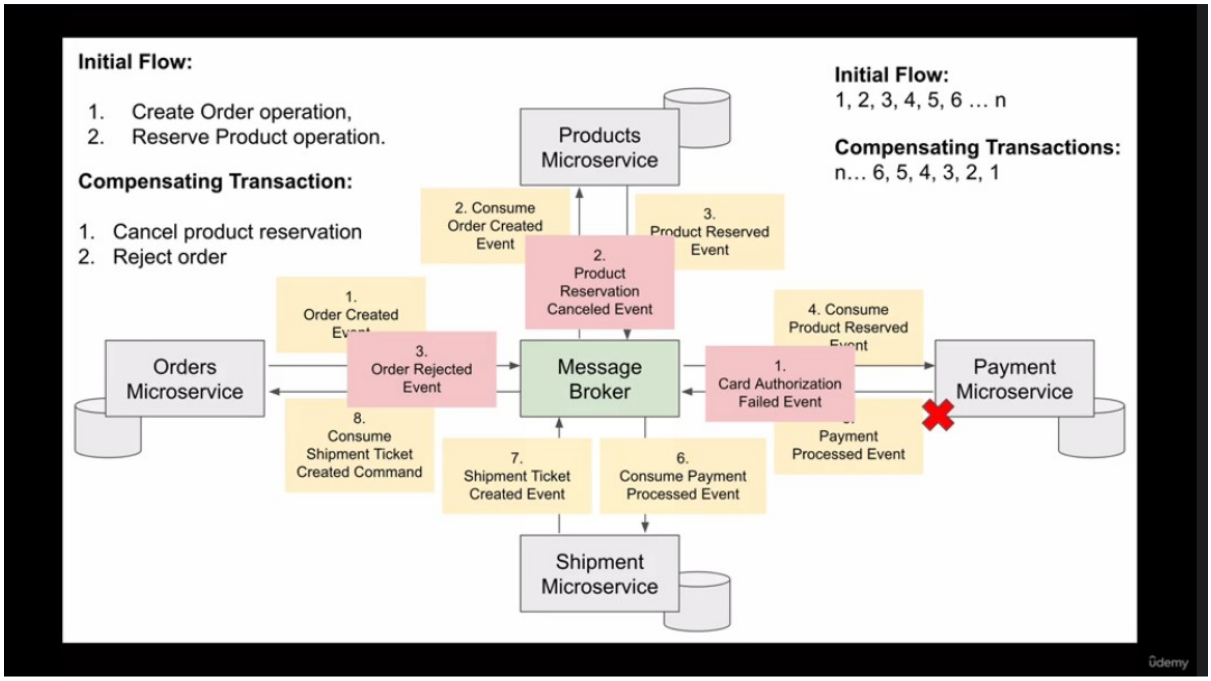
→ Deduplication / Event IDs

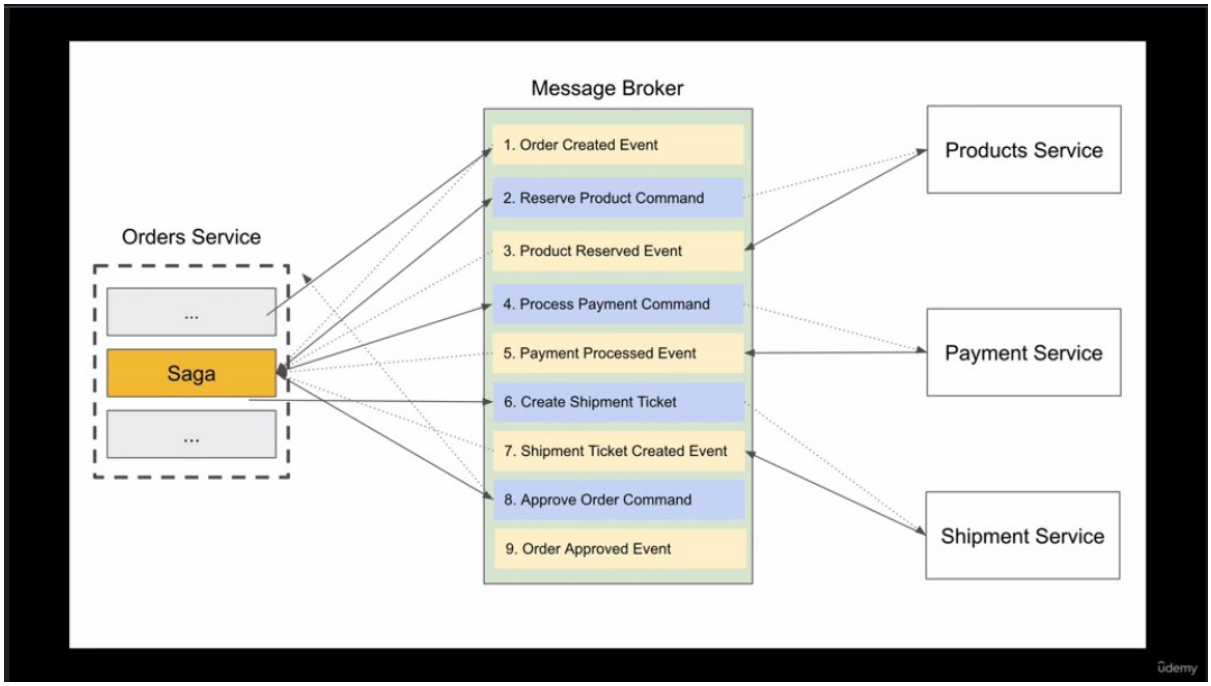
Log Tracing for Kafka & JPA Transaction

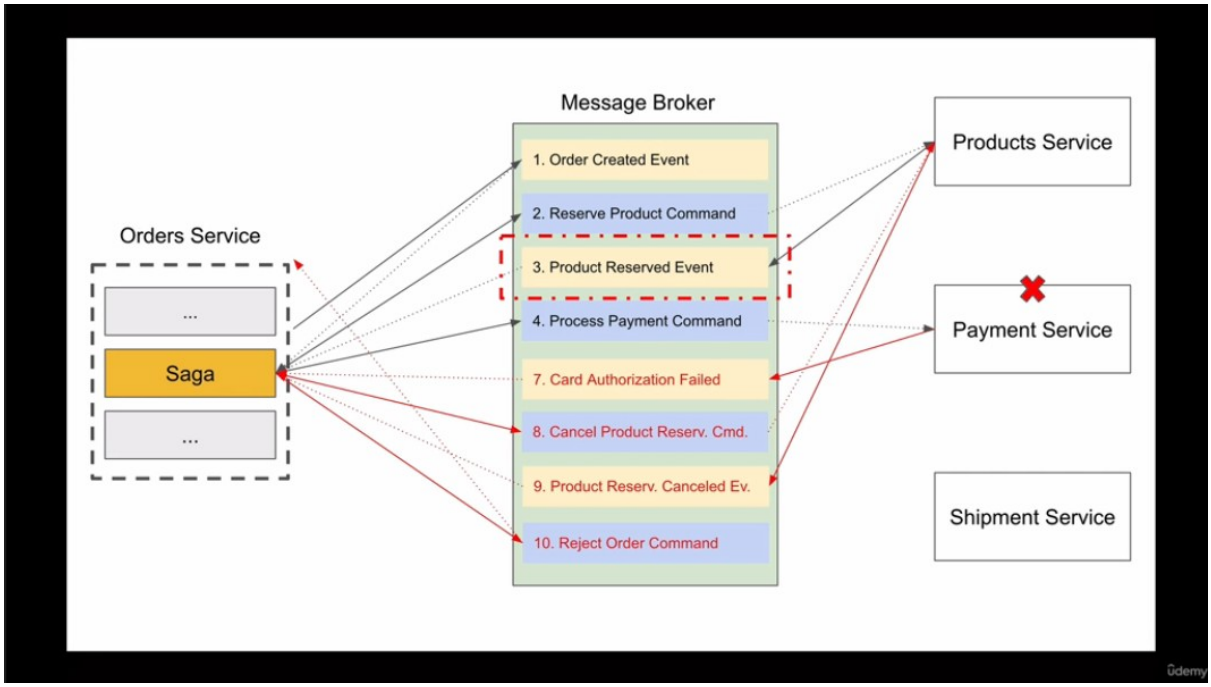


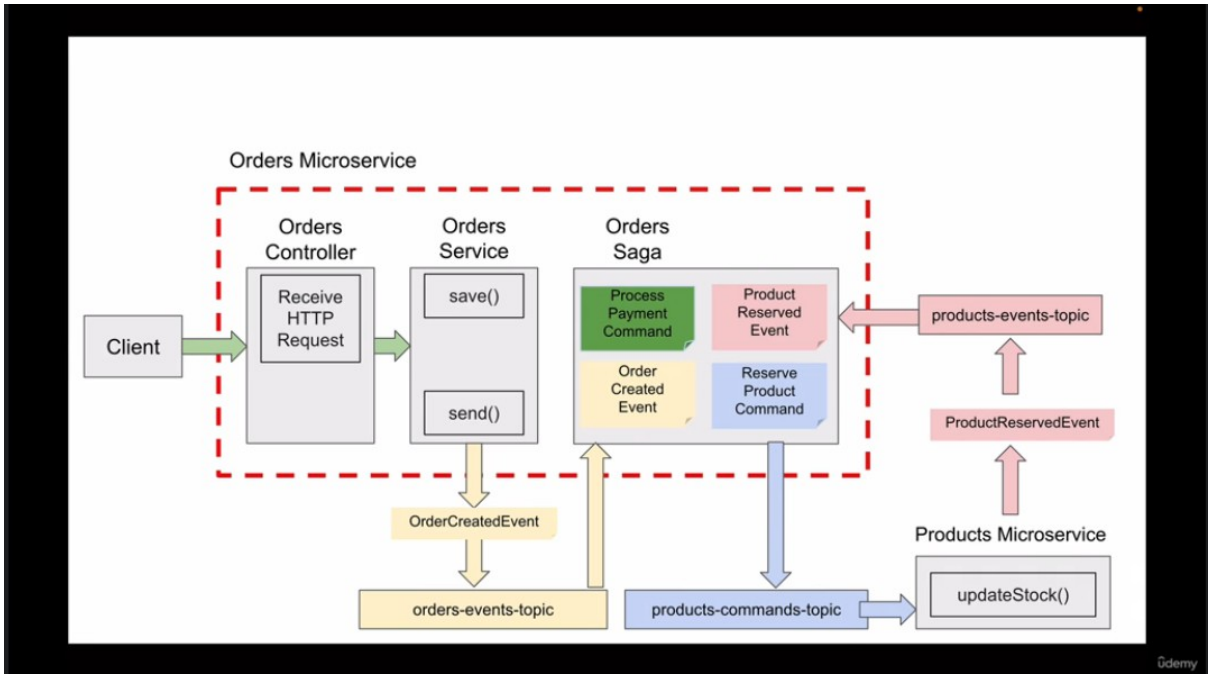

Begin Transaction
1. Create new order,
2. Reserve product in stock,
3. Process payment,
4. Create shipment ticket,
End transaction
Products Microservice
2. Consume Order Created Event
3. Product Reserved Event
1. Order Created Event
4. Consume Product Reserved Event
Orders Microservice
Message Broker
Payment Microservice
8. Consume Shipment Ticket Created Command
7. Shipment Ticket Created Event
6. Consume Payment Processed Event
5. Payment Processed Event
Shipment Microservice
Udemy


Products Microservice
2. Consume Order Created Event
3. Product Reserved Event
2. Product Reservation Canceled Event
1. Order Created Event
4. Consume Product Reserved Event
Orders Microservice
3. Order Rejected Event
Message Broker
1. Card Authorization Failed Event
Payment Microservice
8. Consume Shipment Ticket Created Command
7. Shipment Ticket Created Event
6. Consume Payment Processed Event
Payment Processed Event
Shipment Microservice
Udemy


Initial Flow:
1. Create Order operation,
2. Reserve Product operation.
Compensating Transaction:
1. Cancel product reservation
2. Reject order
Initial Flow:
1, 2, 3, 4, 5, 6 … n
Compensating Transactions:
n… 6, 5, 4, 3, 2, 1
Products Microservice
2. Consume Order Created Event
3. Product Reserved Event
2. Product Reservation Canceled Event
1. Order Created Event
Orders Microservice
3. Order Rejected Event
Message Broker
1. Card Authorization Failed Event
4. Consume Product Reserved Event
Payment Microservice
Payment Processed Event
8. Consume Shipment Ticket Created Command
7. Shipment Ticket Created Event
6. Consume Payment Processed Event
Shipment Microservice
Udemy


Message Broker
1. Order Created Event
2. Reserve Product Command
3. Product Reserved Event
4. Process Payment Command
5. Payment Processed Event
6. Create Shipment Ticket
7. Shipment Ticket Created Event
8. Approve Order Command
9. Order Approved Event
Orders Service
...
Saga
...
Products Service
Payment Service
Shipment Service
Udemy


Message Broker
1. Order Created Event
2. Reserve Product Command
3. Product Reserved Event
4. Process Payment Command
7. Card Authorization Failed
8. Cancel Product Reserv. Cmd.
9. Product Reserv. Canceled Ev.
10. Reject Order Command
Orders Service
...
Saga
...
Products Service
Payment Service
Shipment Service
Udemy

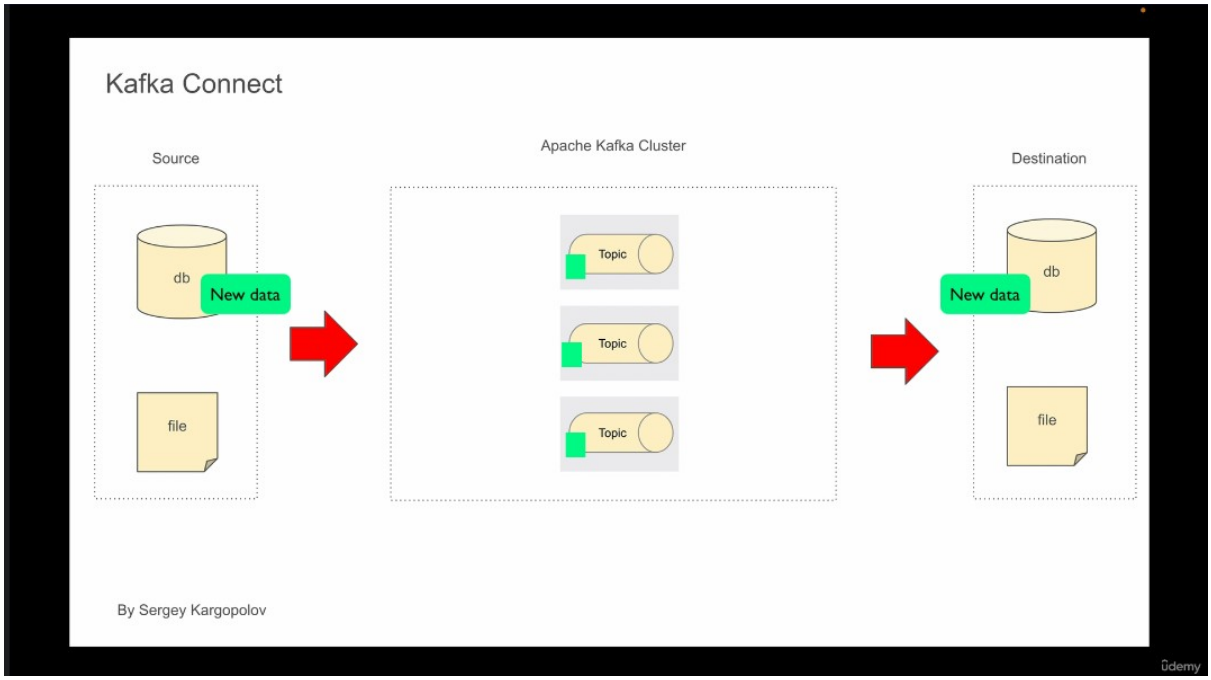


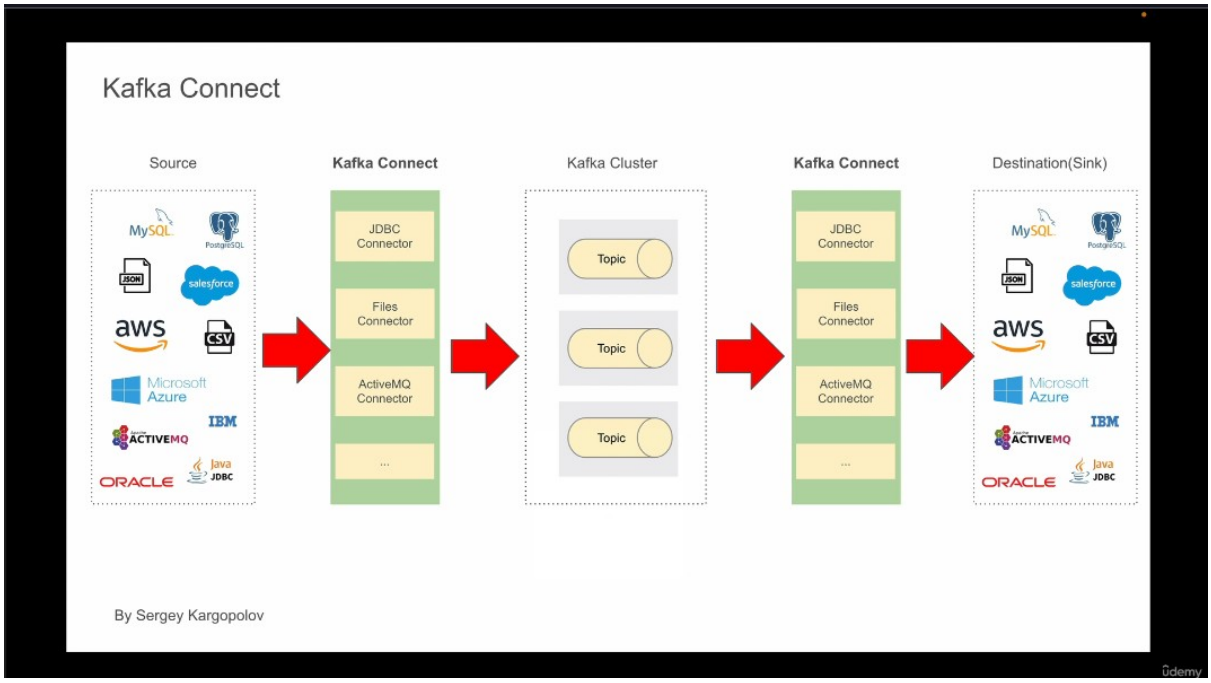
What is Kafka Connect?

Apache Kafka Connect is a free tool that helps you move data between Apache Kafka and other systems, like databases or file storage.

By Sergey Kargopolov

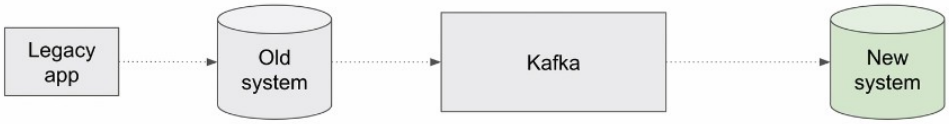
udemy


Kafka Connect
Source
Apache Kafka Cluster
Destination
db
New data
file
Topic
Topic
Topic
db
New data
file
By Sergey Kargopolov
udemy


Kafka Connect
Source
Kafka Connect
Kafka Cluster
Kafka Connect
Destination(Sink)
JDBC Connector
Files Connector
ActiveMQ Connector
...
Topic
Topic
Topic
By Sergey Kargopolov
udemy

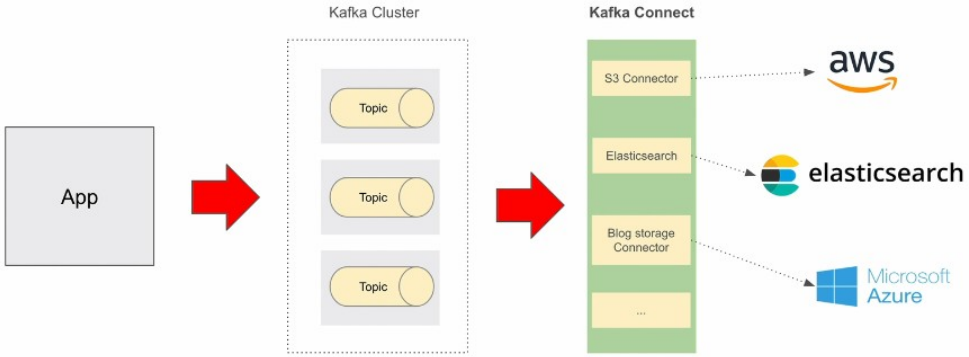
When to use Kafka Connect?

- When you need to continuously move data between Kafka and external systems
- When you want to avoid writing custom integration code
- When you want real-time, reliable pipelines between systems.



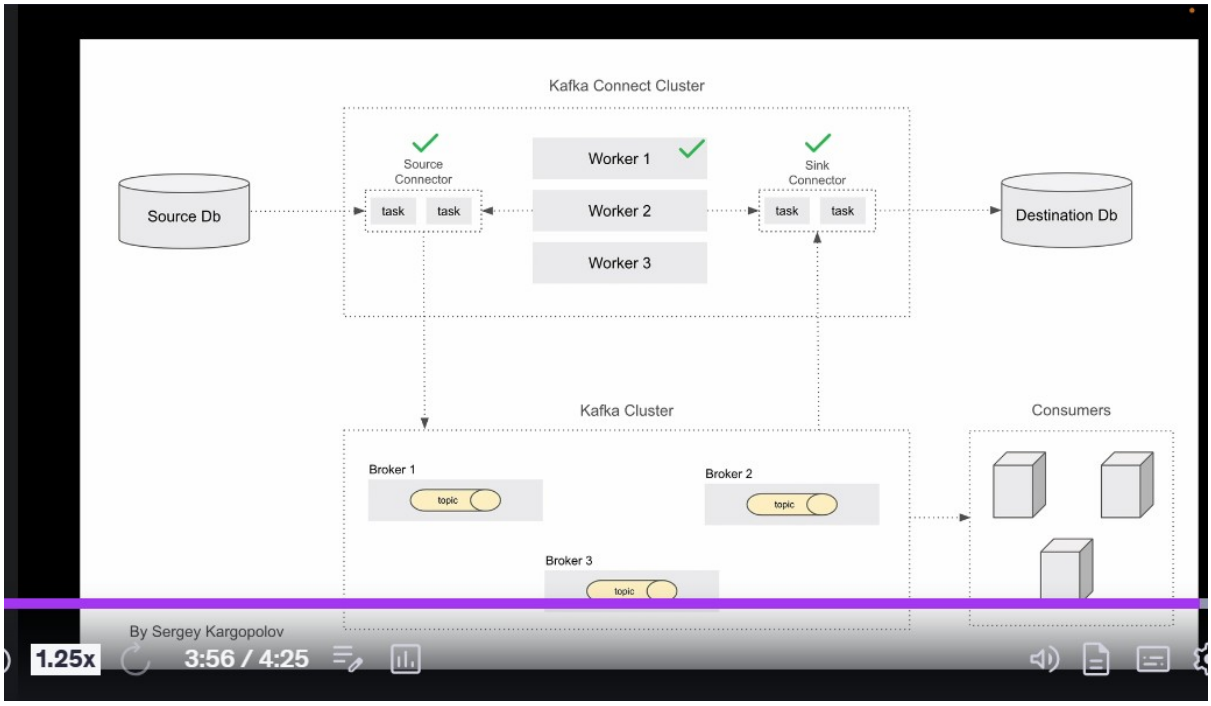
By Sergey Kargopolov

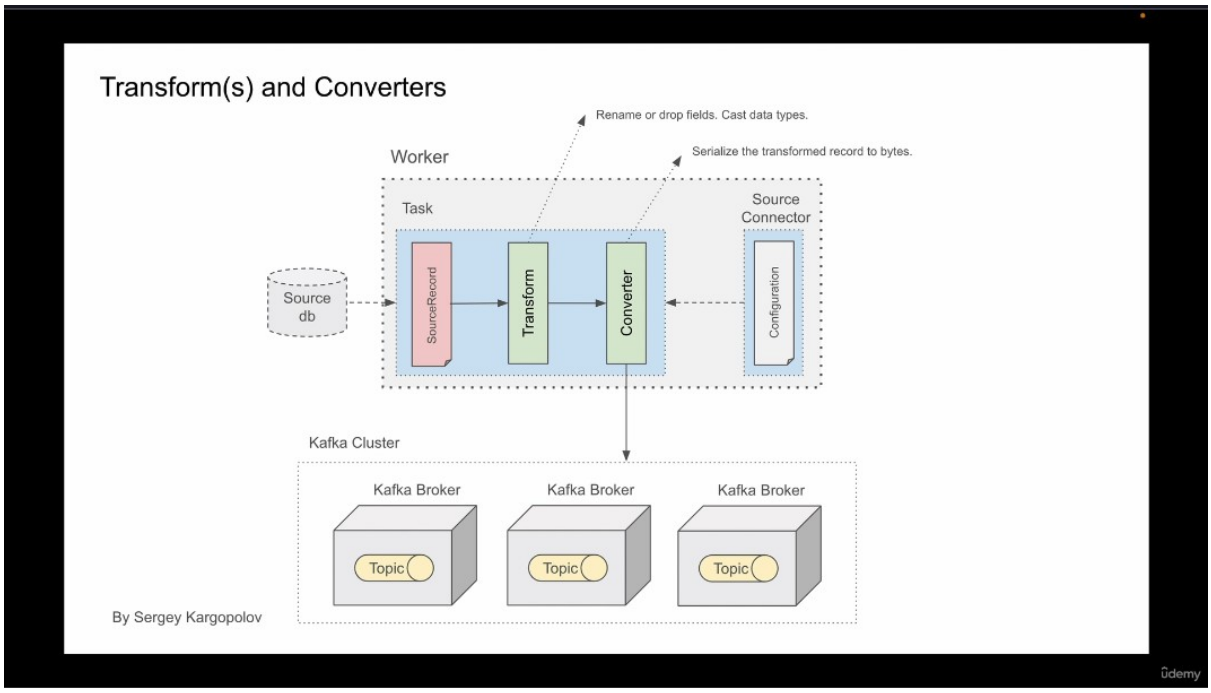
Udemy



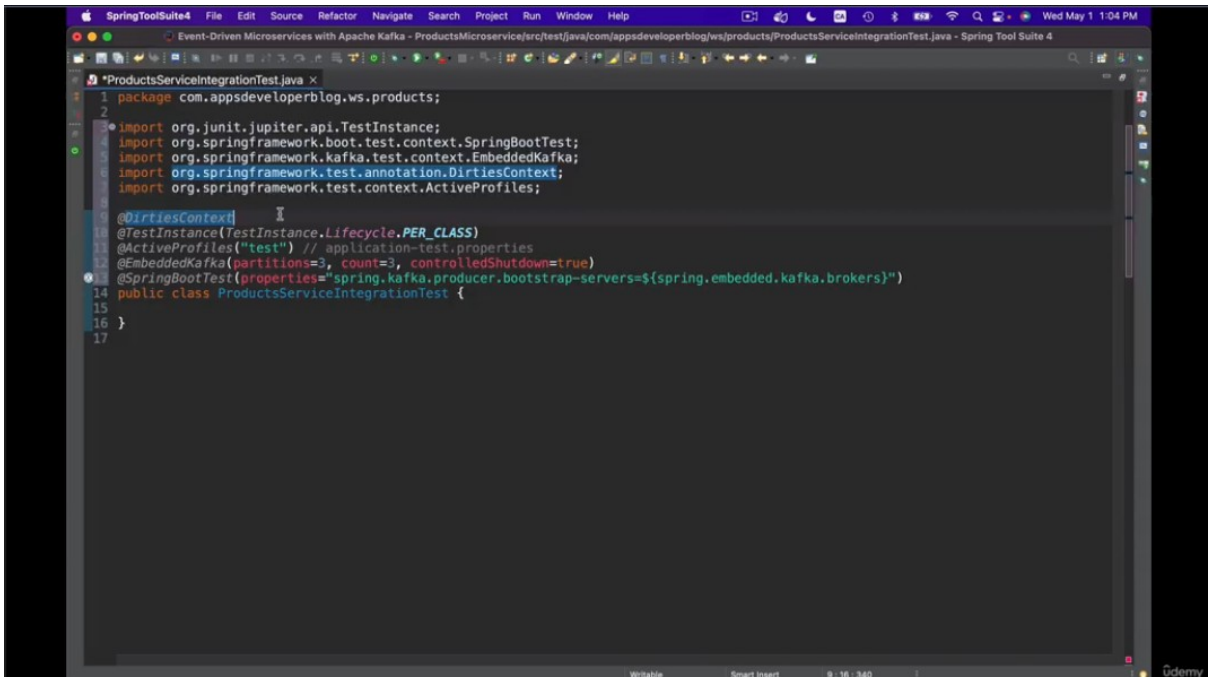
By Sergey Kargopolov

Udemy


Kafka Connect Cluster
Source Db
Source Connector
task
task
Worker 1
Worker 2
Worker 3
Sink Connector
task
task
Destination Db
Kafka Cluster
Broker 1
topic
Broker 2
topic
Broker 3
topic
Consumers
By Sergey Kargopolov
1.25x
3:56 / 4:25


Transform(s) and Converters
Rename or drop fields. Cast data types.
Serialize the transformed record to bytes.
Worker
Task
Source Connector
Source db
SourceRecord
Transform
Converter
Configuration
Kafka Cluster
Kafka Broker
Kafka Broker
Kafka Broker
Topic
Topic
Topic
By Sergey Kargopolov
udemy

TEST CASES


SpringToolSuite4 File Edit Source Refactor Navigate Search Project Run Window Help
Wed May 1 1:04 PM
Event-Driven Microservices with Apache Kafka - ProductsMicroservice/src/test/java/com/appsdeveloperblog/ws/products/ProductsServiceIntegrationTest.java - Spring Tool Suite 4
*ProductsServiceIntegrationTest.java
package com.appsdeveloperblog.ws.products;
import org.junit.jupiter.api.TestInstance;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.kafka.test.context.EmbeddedKafka;
import org.springframework.test.annotation.DirtiesContext;
import org.springframework.test.context.ActiveProfiles;
@DirtiesContext
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
@ActiveProfiles("test") // application-test.properties
@EmbeddedKafka(partitions=3, count=3, controlledShutdown=true)
@SpringBootTest(properties="spring.kafka.producer.bootstrap-servers=${spring.embedded.kafka.brokers}")
public class ProductsServiceIntegrationTest {
}